

Classification

Data Science

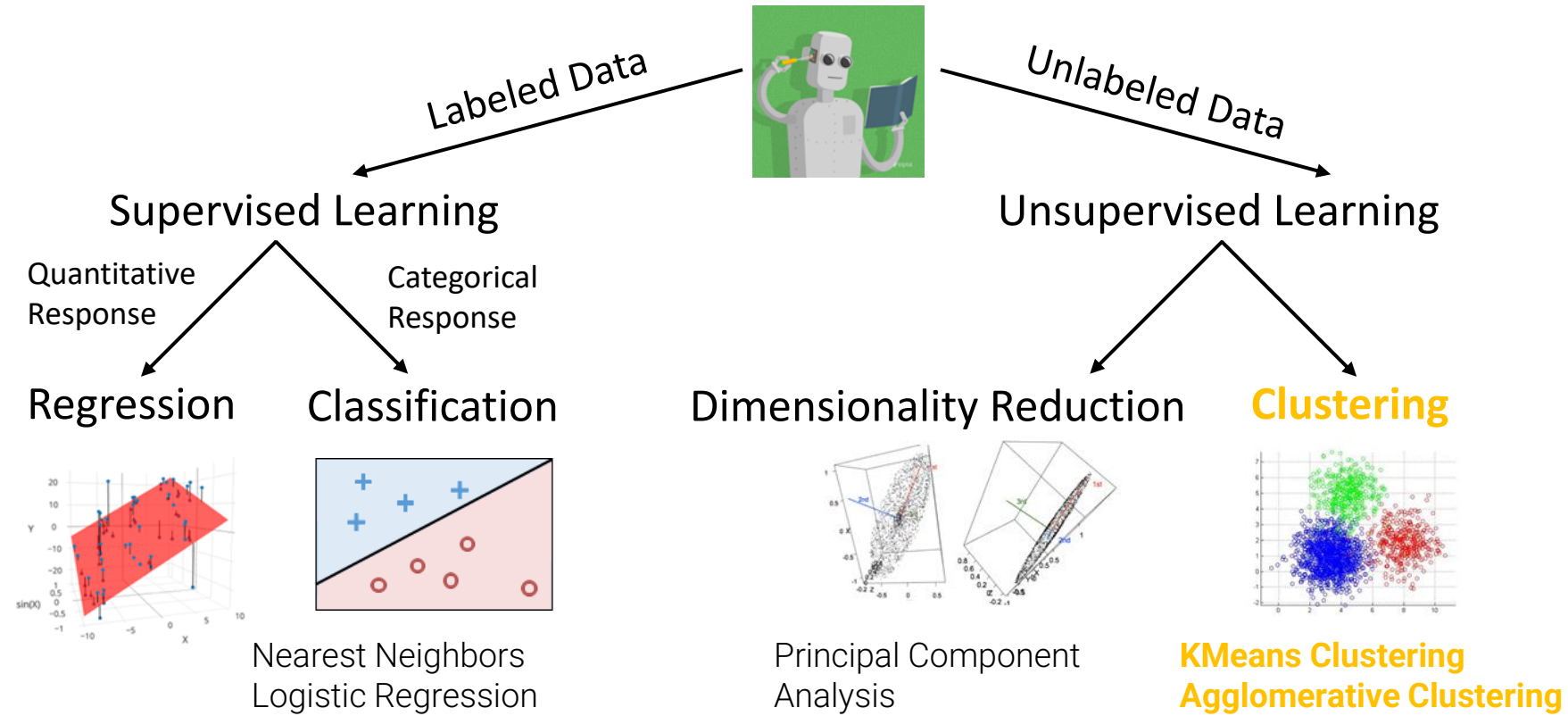
COMP5122M

Lecturer: Duygu Sarikaya

What will you learn?

- Define what classification is
- Classify Iris Species using k-nearest neighbors
- Classify Iris Species using Decision Trees
- Building Decision Trees
- Entropy and Defining a Best Feature

Recap



Recap

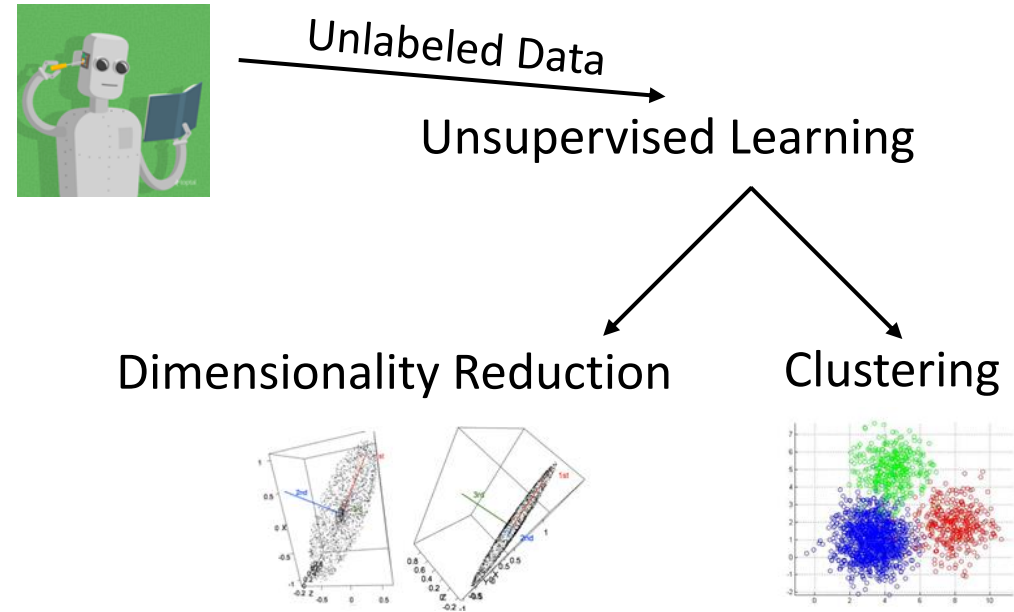
In “Unsupervised Learning”:

- Goal is to identify patterns in **unlabeled** data.
 - We do not have input/output pairs.
 - Note: Sometimes we may have labels, but we’re just choosing to ignore them (e.g. PCA on labeled data).

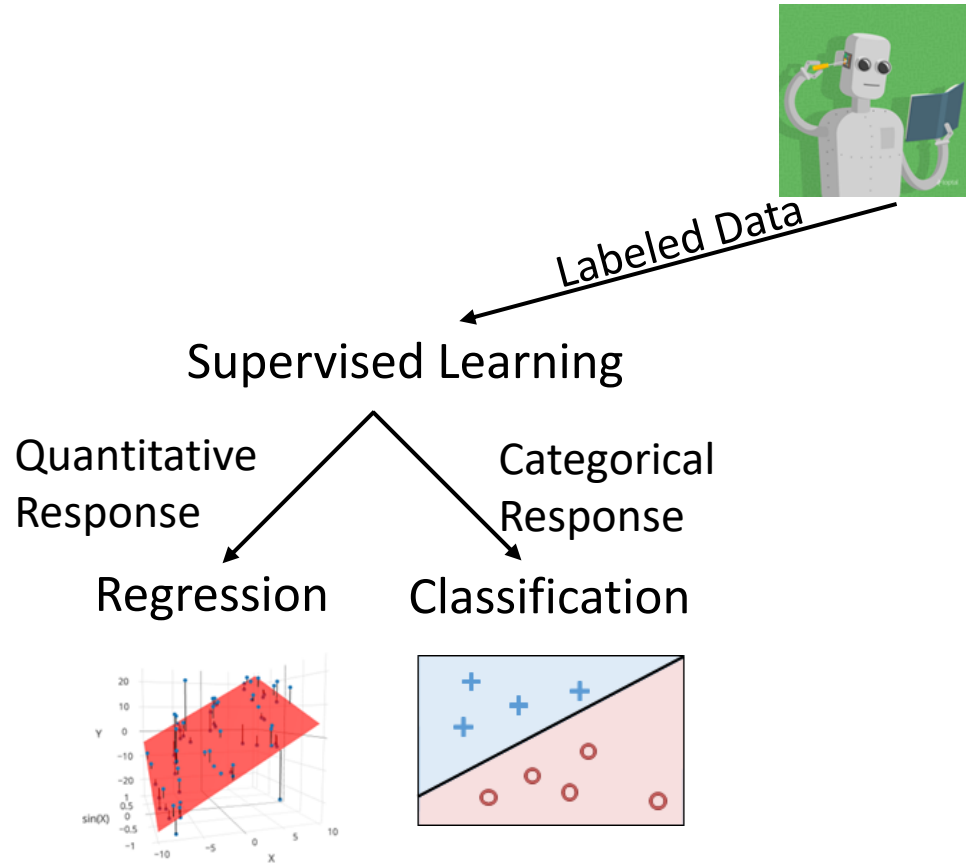
Dimensionality Reduction.

- PCA.

Clustering.

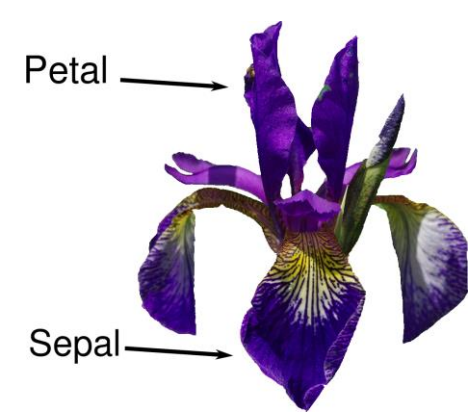


Recap

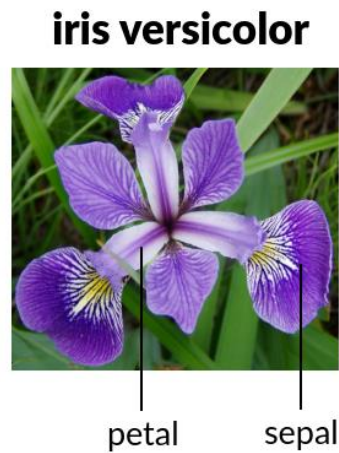


In “Supervised Learning”:

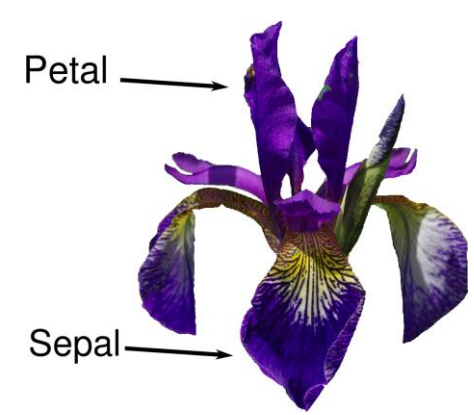
- Goal is to create a function that maps inputs to outputs.
- Model is learned from example input/output pairs. Each pair consists of:
 - Input vector.
 - Output value (**label**).
- Regression: Output value is quantitative.
- Classification: Output value is categorical.



A First Application: Classifying Iris Species

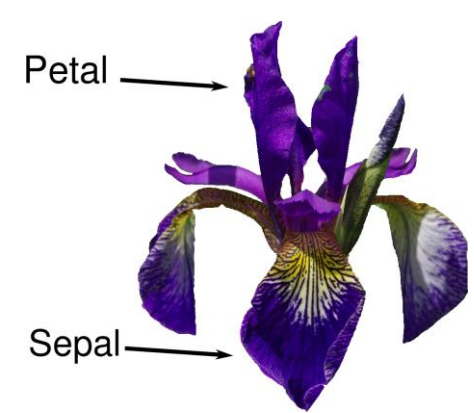


- A botanist is interested in distinguishing the species of some iris flowers that she has found. She has collected some measurements associated with each iris: the length and width of the petals and the length and width of the sepals, all measured in centimeters
- Our goal is to build a machine learning model that can learn from the measurements of these irises whose species is known, so that we can predict the species for a new iris.



A First Application: Classifying Iris Species

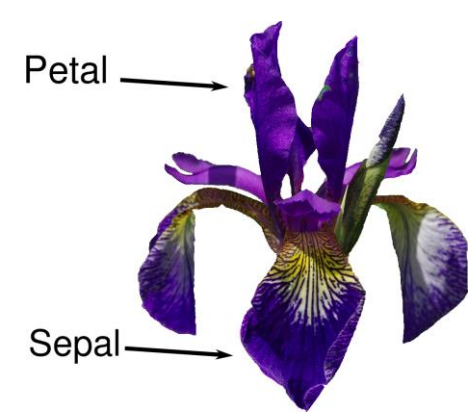
- Because we have measurements for which we know the correct species of iris, this is a ***supervised learning*** problem.
- We want to predict one of several options (the species of iris). This is an example of a ***classification*** problem.
- The possible outputs (different species of irises) are called ***classes***.
- Every iris in the dataset belongs to one of three classes, so this problem is a ***three-class classification problem***.
- The desired output for a single data point (an iris) is the species of this flower. For a particular data point, the species it belongs to is called its ***label***.



A First Application: Classifying Iris Species

- Iris Plants Database
- =====
- Notes
- ----
- Data Set Characteristics:
- :Number of Instances: 150 (50 in each of three classes)
- :Number of Attributes: 4 numeric, predictive attributes
- ...
- ----

sepal_length	sepal_width	petal_length	petal_width	species
5.5	2.5	4.0	1.3	versicolor
6.4	2.9	4.3	1.3	versicolor
4.8	3.4	1.6	0.2	setosa
5.3	3.7	1.5	0.2	setosa
6.7	2.5	5.8	1.8	virginica



A First Application: Classifying Iris Species

We want to build a machine learning model from this data that can predict the species of iris for a new set of measurements.

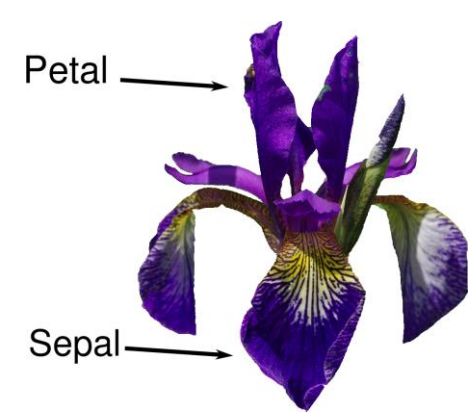
How to measure success? Training and Testing Data

To assess the model's performance, we show it new data (data that it hasn't seen before) for which we have labels.

This is usually done by splitting the labeled data we have collected (here, our 150 flower measurements) into two parts.

One part of the data is used to build our machine learning model, and is called the ***training data*** or ***training set***.

The rest of the data will be used to assess how well the model works; this is called the ***test data***, ***test set***, or ***hold-out set***.



A First Application: Classifying Iris Species

Shuffling the dataset:

Using a test set containing only one of the three classes would not tell us much about how well our model generalizes, so we shuffle our data to make sure the test data contains data from all classes.

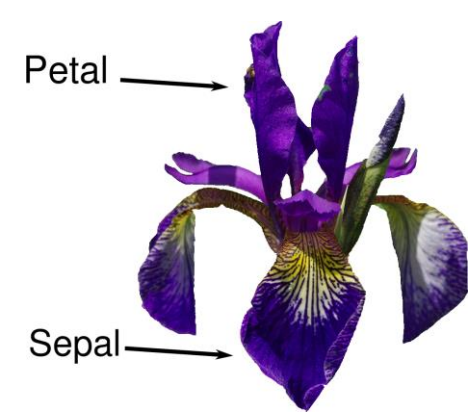
Splitting the dataset

We can use 75% of the rows in the data as the ***training set***, together with the corresponding labels for this data. The remaining 25% of the data, together with the remaining labels, is declared as the ***test set***.

%75, %25

%80, %20

%70, %30



A First Application: Classifying Iris Species

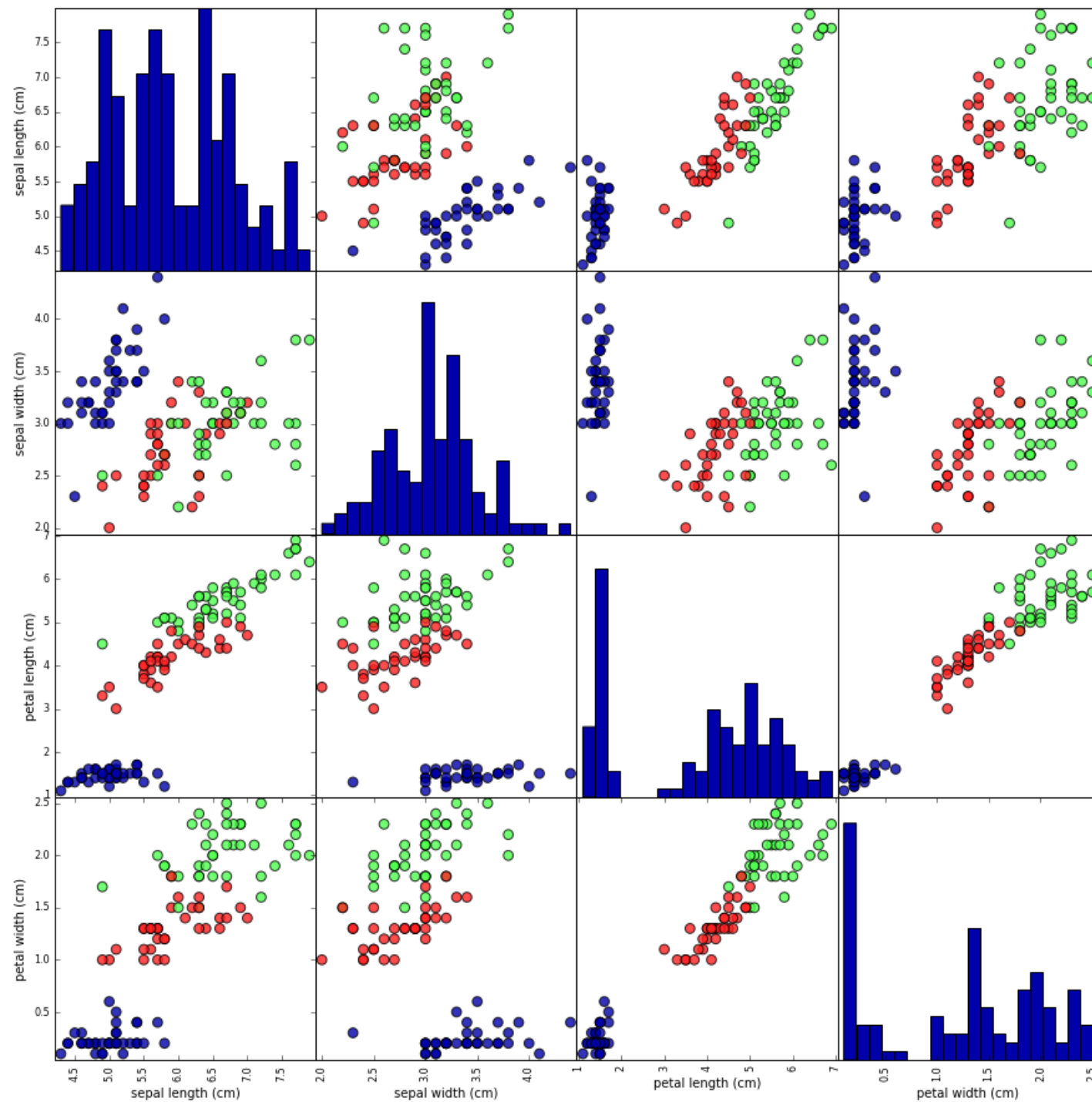
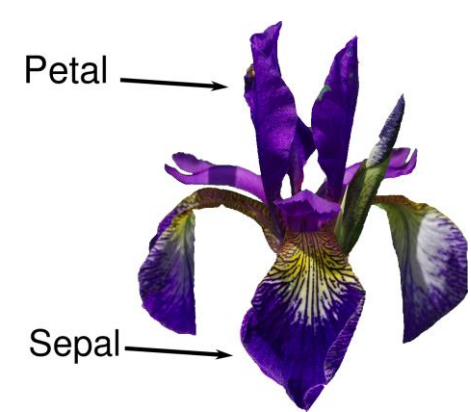
The output of the `train_test_split` function (scikit-learn) is `X_train`, `X_test`, `y_train`, and `y_test`, which are all NumPy arrays. `X_train` contains 75% of the rows of the dataset, and `X_test` contains the remaining 25%:

```
print("X_train shape: {}".format(X_train.shape))  
print("y_train shape: {}".format(y_train.shape))
```

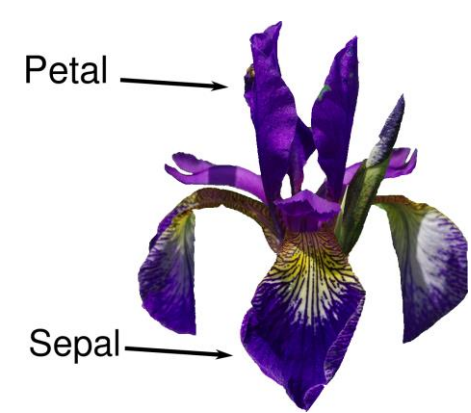
```
X_train shape: (112, 4)  
y_train shape: (112,)
```

```
print("X_test shape: {}".format(X_test.shape))  
print("y_test shape: {}".format(y_test.shape))
```

```
X_test shape: (38, 4)  
y_test shape: (38,)
```



Three classes seem to be relatively well separated using the sepal and petal measurements.



A First Application: Classifying Iris Species

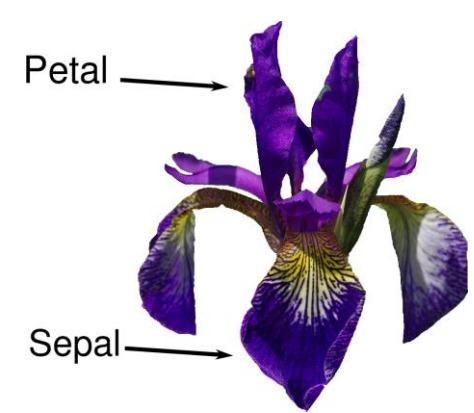
Building Your First Model: k-Nearest Neighbors:

There are many classification algorithms in scikit-learn that we could use. Here we will use a *k*-nearest neighbors classifier.

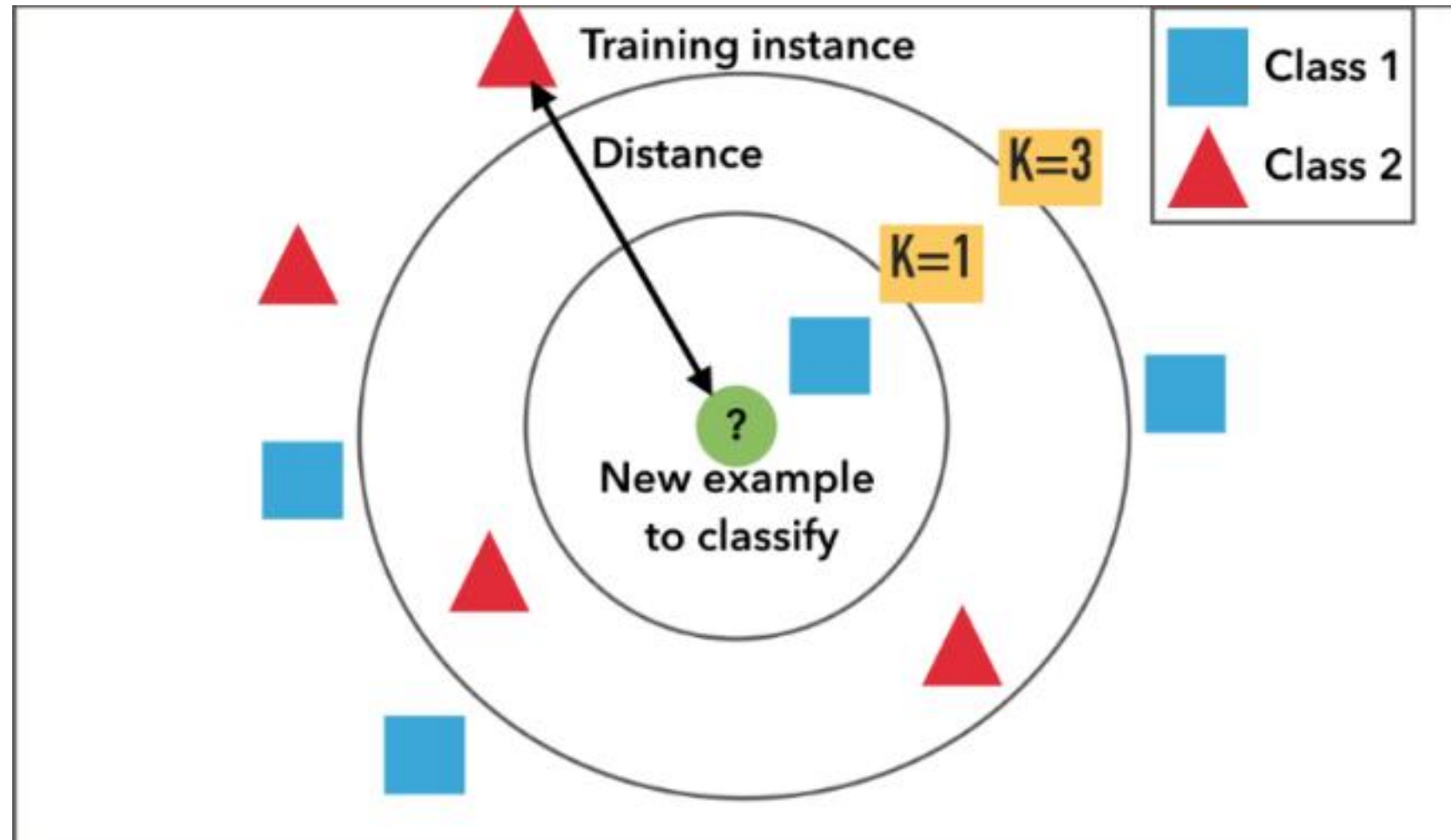
- Stores the training set.
- To make a prediction for a new data point, the algorithm finds the point in the training set that is closest to the new point.
- Then it assigns the label of this training point to the new data point.

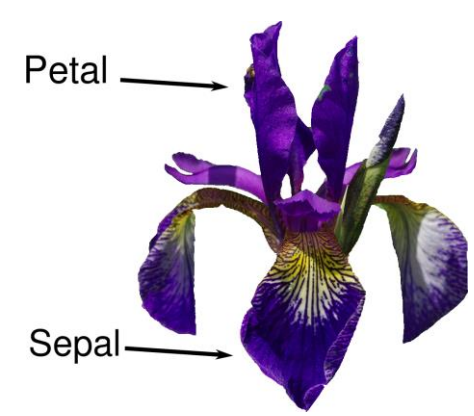
The *k* in *k*-nearest neighbors signifies that instead of using only the closest neighbor to the new data point, we can consider any fixed number *k* of neighbors in the training (for example, the closest three or five neighbors).

Then, we can make a prediction using the majority class among these neighbors.



A First Application: Classifying Iris Species

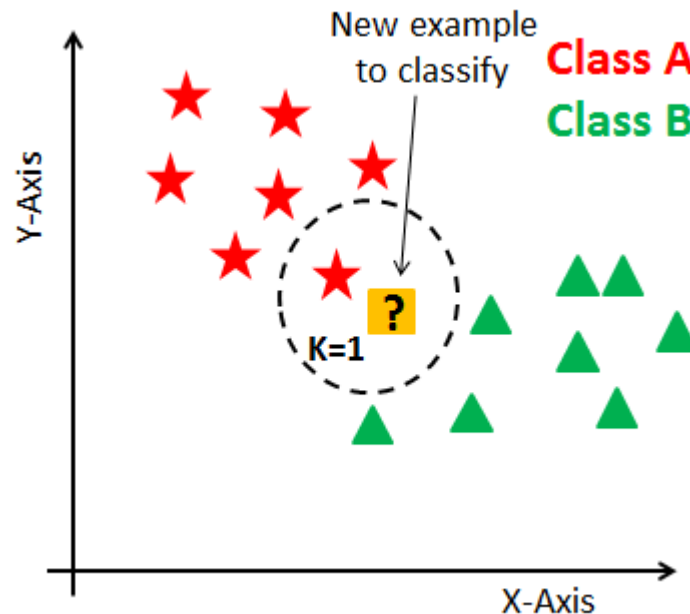


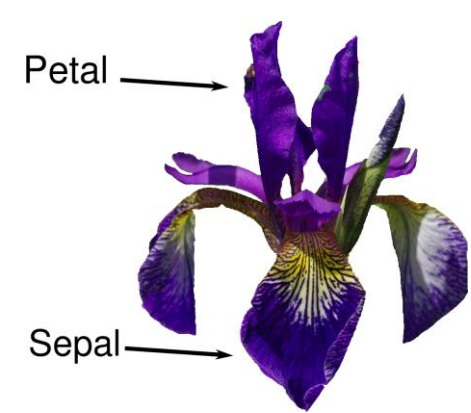


A First Application: Classifying Iris Species

In K-NN, K is the number of nearest neighbors. The number of neighbors is the core deciding factor. **K is generally an odd number if the number of classes is 2.** When $K=1$, then the algorithm is known as the nearest neighbor algorithm. This is the simplest case.

In the below figure, suppose yellow colored “?” let's say P is the point, for which label needs to predict. First, you find the one closest point to P and then the label of the nearest point assigned to P.

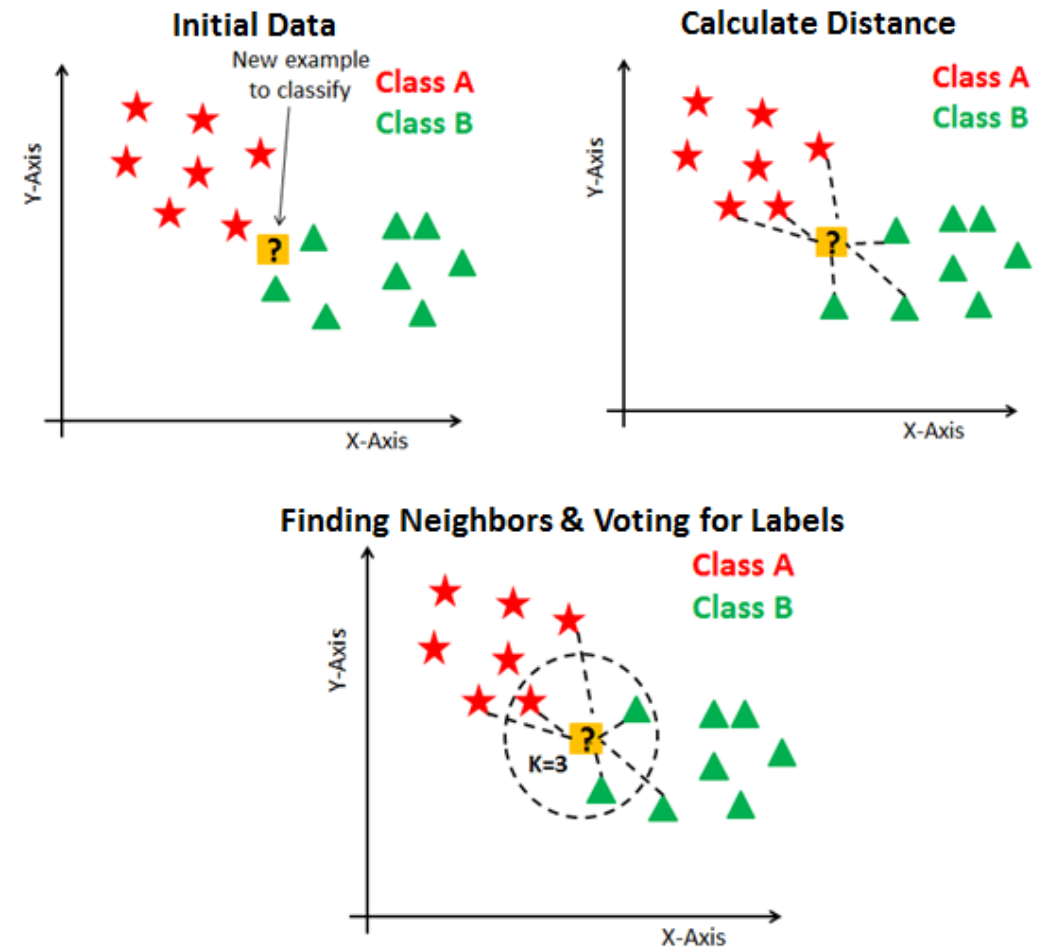


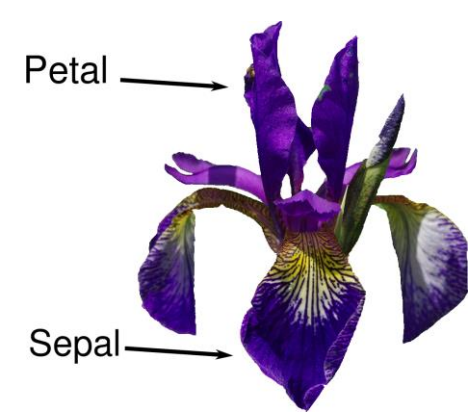


A First Application: Classifying Iris Species

Second, you find the k closest point to P and then classify points by majority vote of its K neighbors. Each object votes for their class and the class with the most votes is taken as the prediction. For finding closest similar points, we find the distance between points using distance measures such as Euclidean distance, Hamming distance, Manhattan distance, and Minkowski distance. The algorithm has the following basic steps:

1. Calculate distance
2. Find closest neighbors
3. Vote for labels





A First Application: Classifying Iris Species

We formulated the task of ***predicting*** which species of iris a particular flower belongs to by using physical measurements of the flower.

We used a dataset of measurements that was annotated by an expert with the correct species to build our model, making this a ***supervised learning task***.

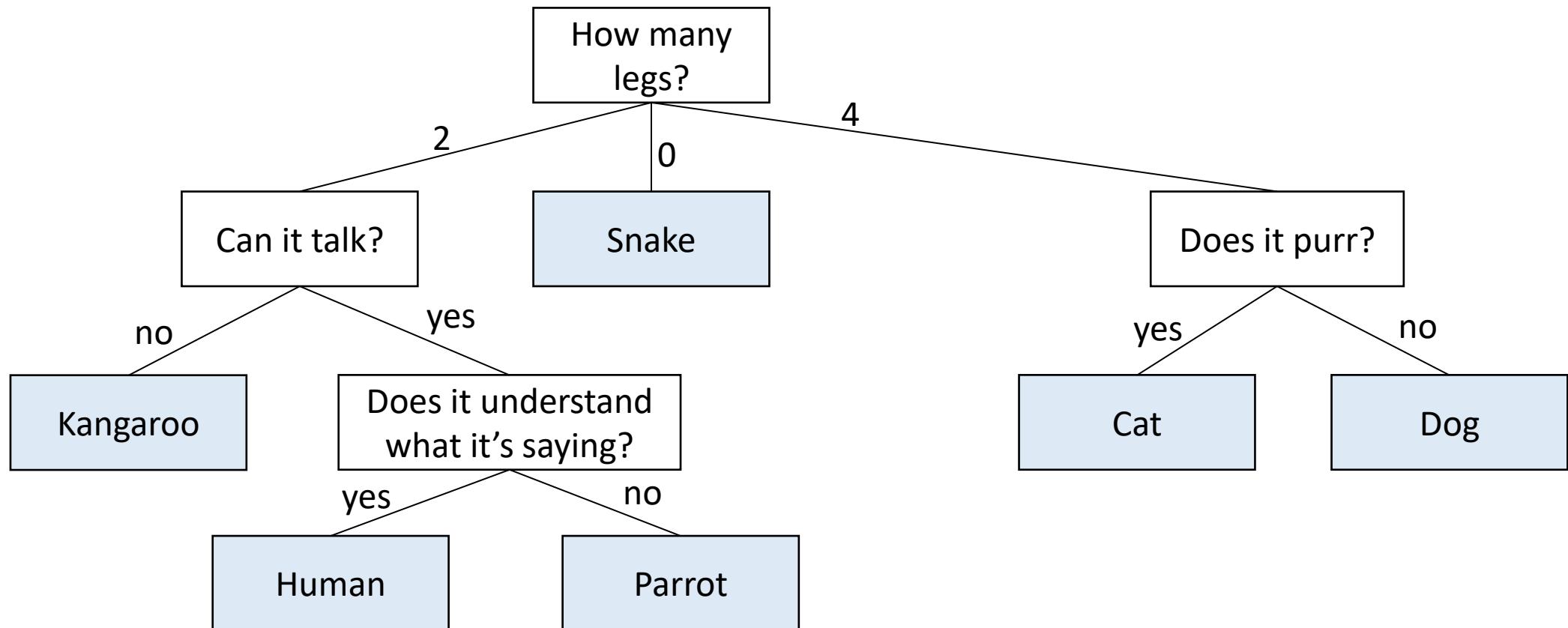
There were three possible species, *setosa*, *versicolor*, or *virginica*, which made the task ***a three-class classification problem***.

The possible species are called *classes* in the classification problem, and the species of a single iris is called its ***label***.

We split our dataset into a ***training set***, to build our model, and a ***test set***, to evaluate how well our model will ***generalize*** to new, previously unseen data.

Decision Trees

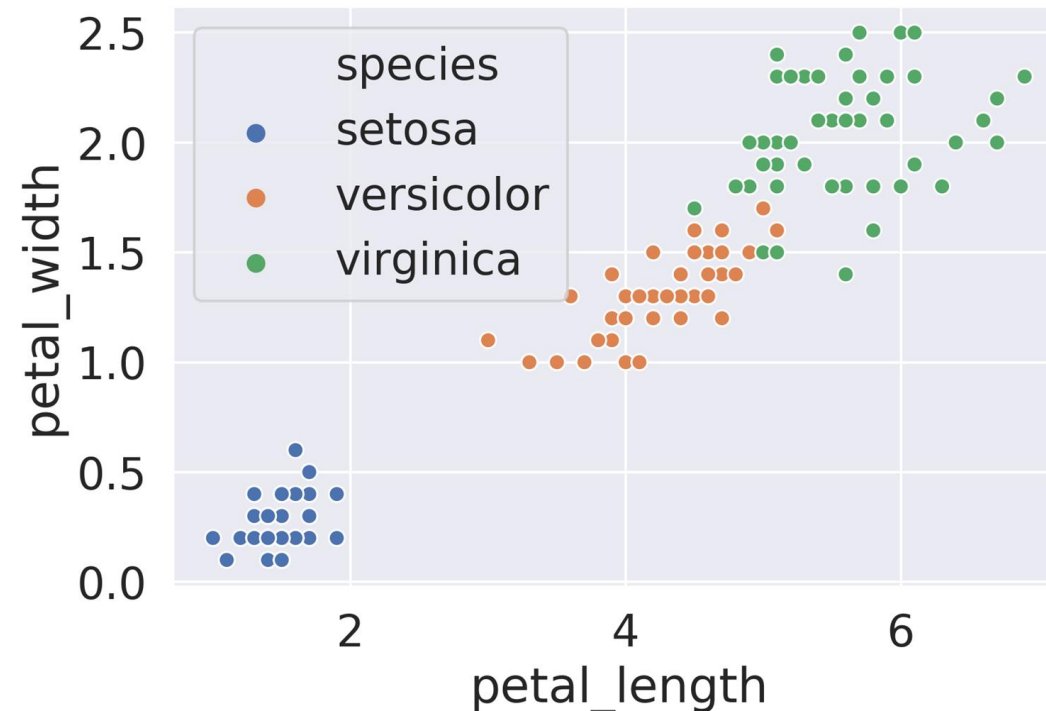
A Decision Tree is a very simple way to classify data. It is simply a tree of questions that must be answered in sequence to yield a predicted classification.



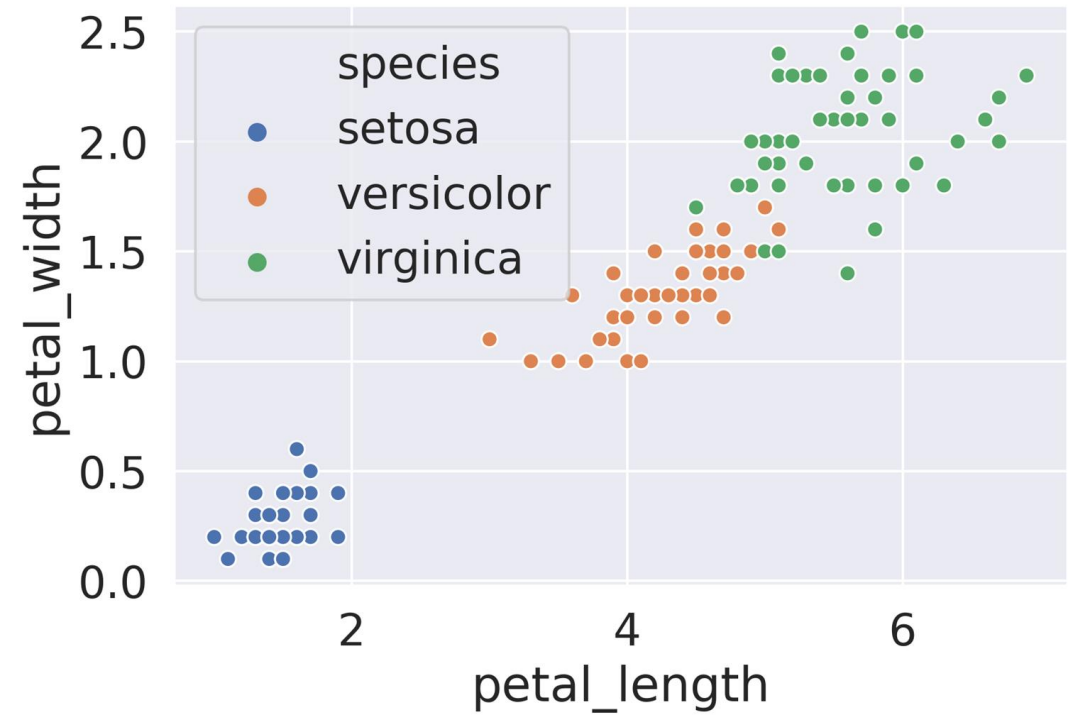
Classifying Iris Species : Using Petal Data Only

The plot below shows the width and length of the petals of each flower, with the species annotated in the form of color.

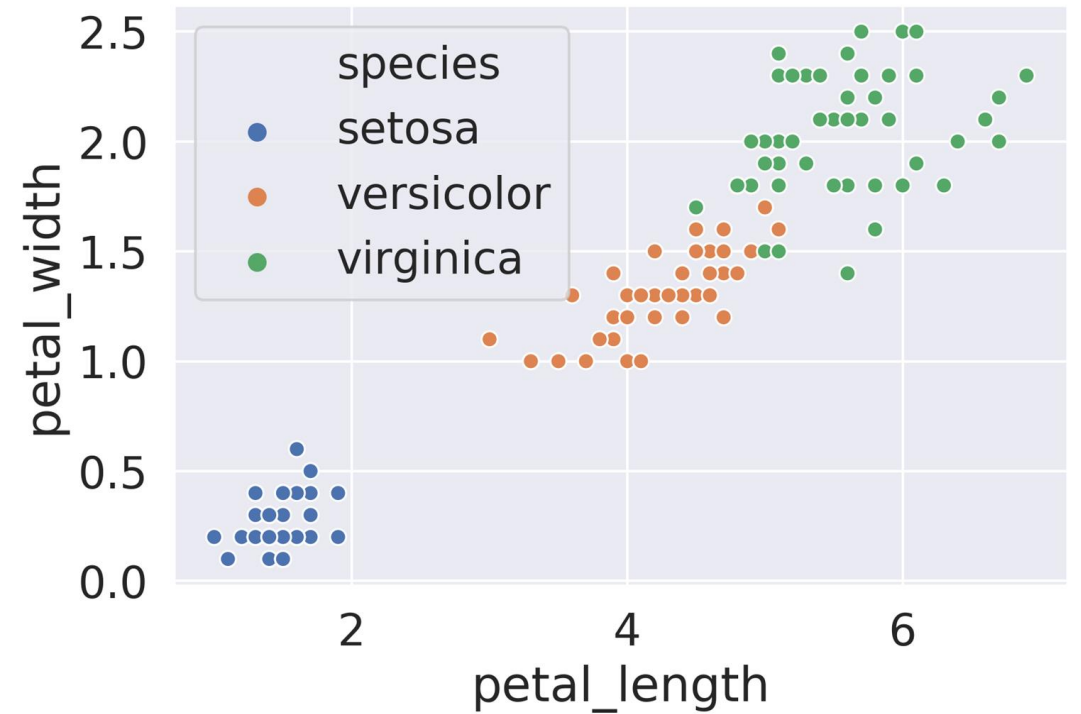
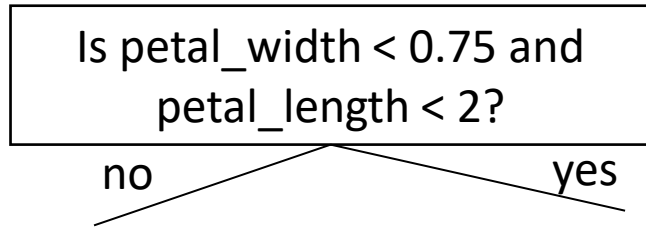
We can build a decision tree manually just by looking at this picture.



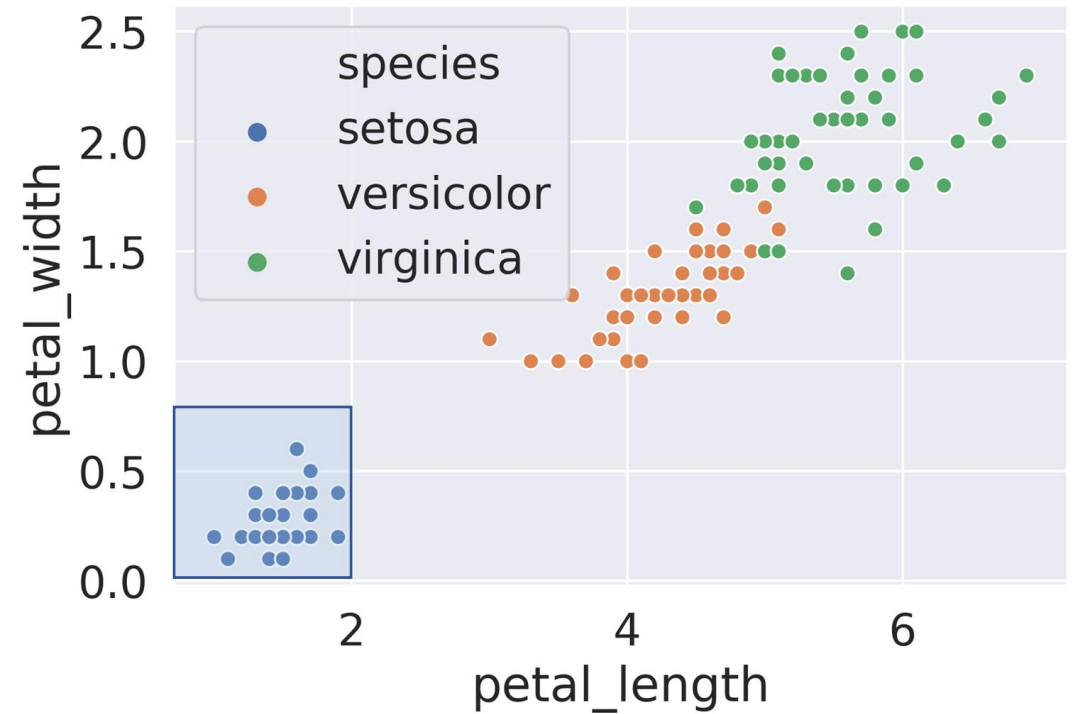
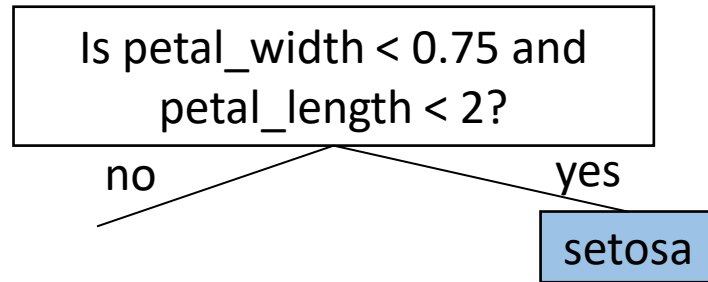
Example: Using Petal Data Only



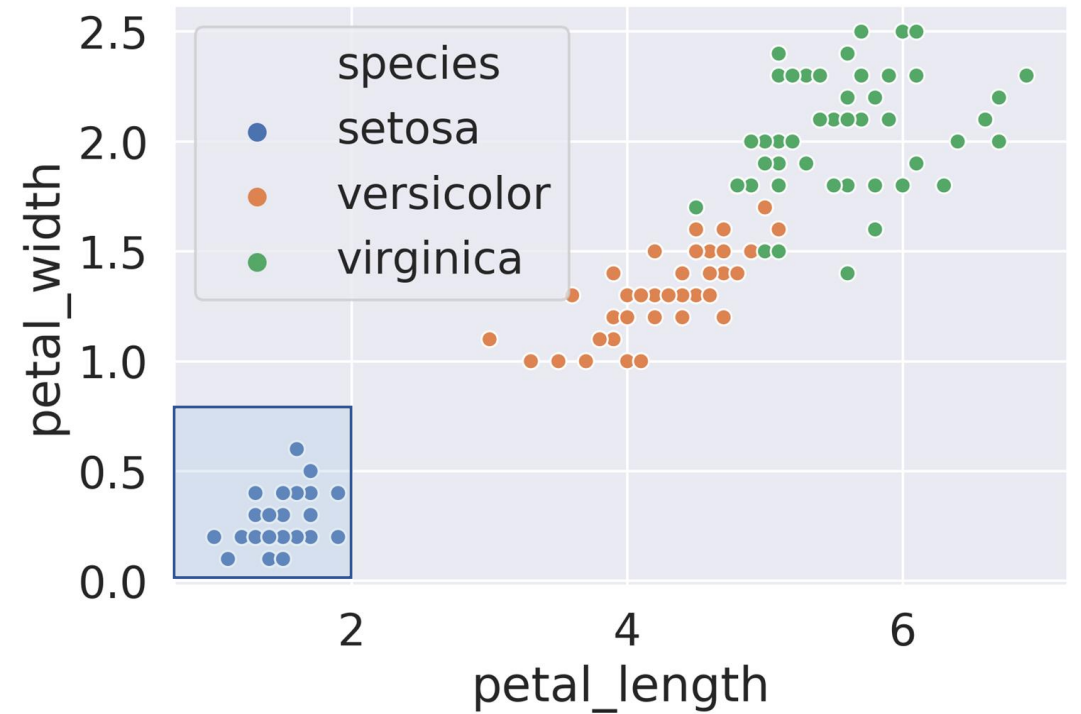
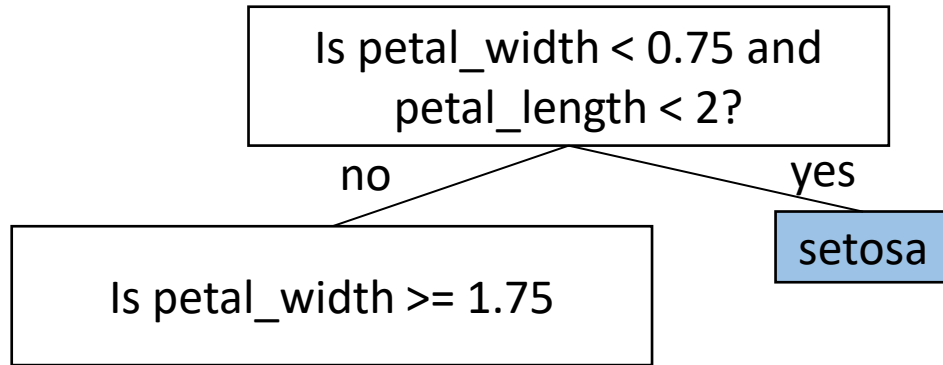
Example: Using Petal Data Only



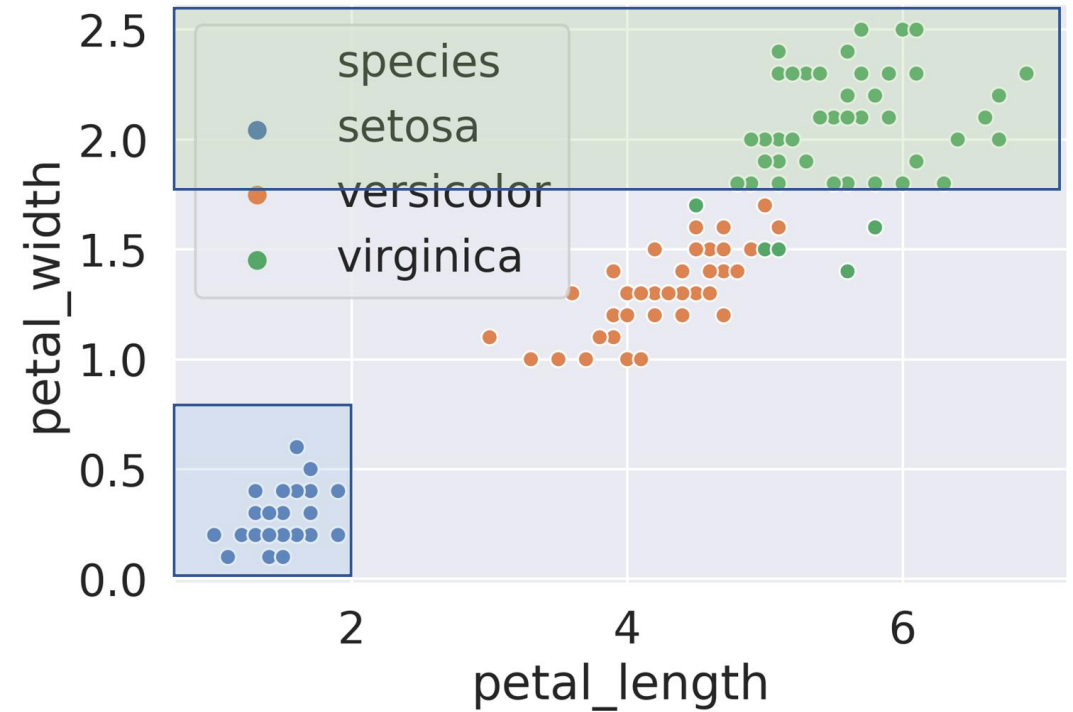
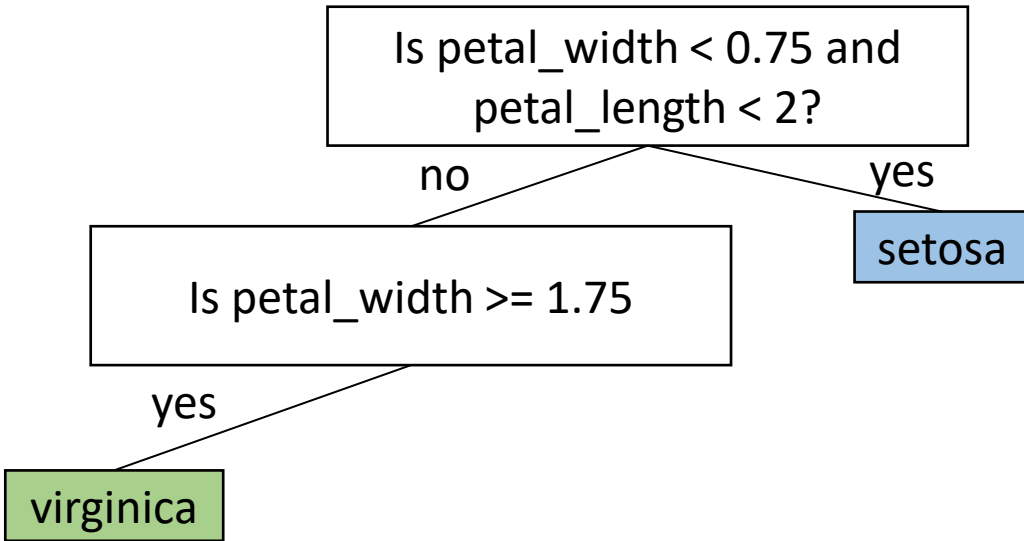
Example: Using Petal Data Only



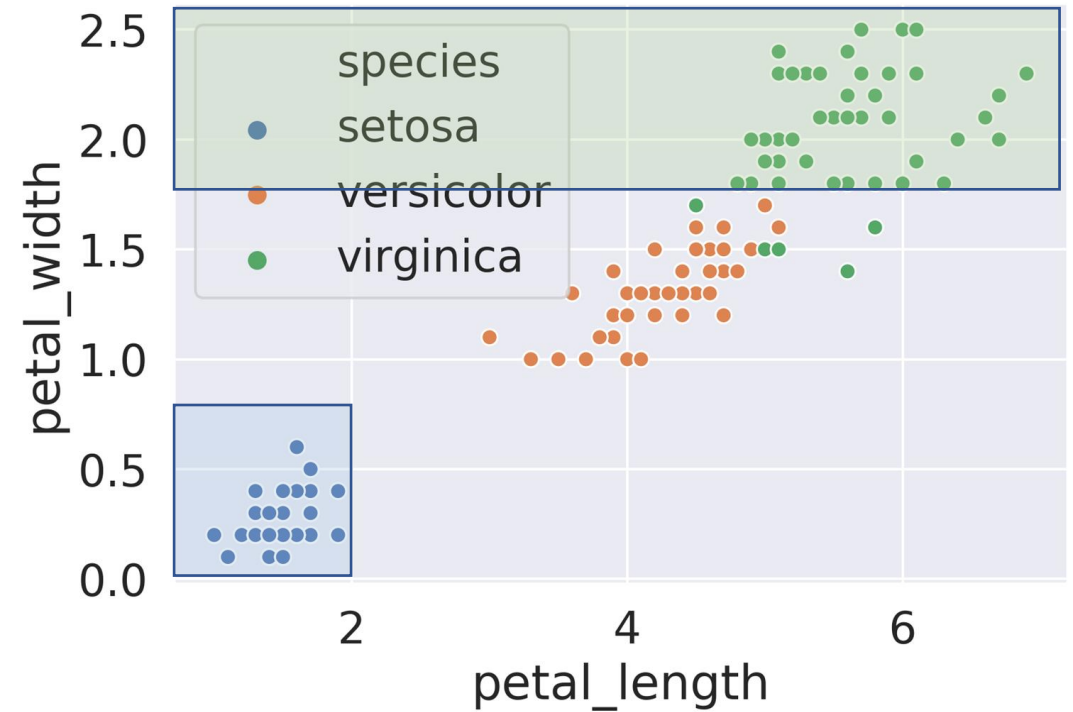
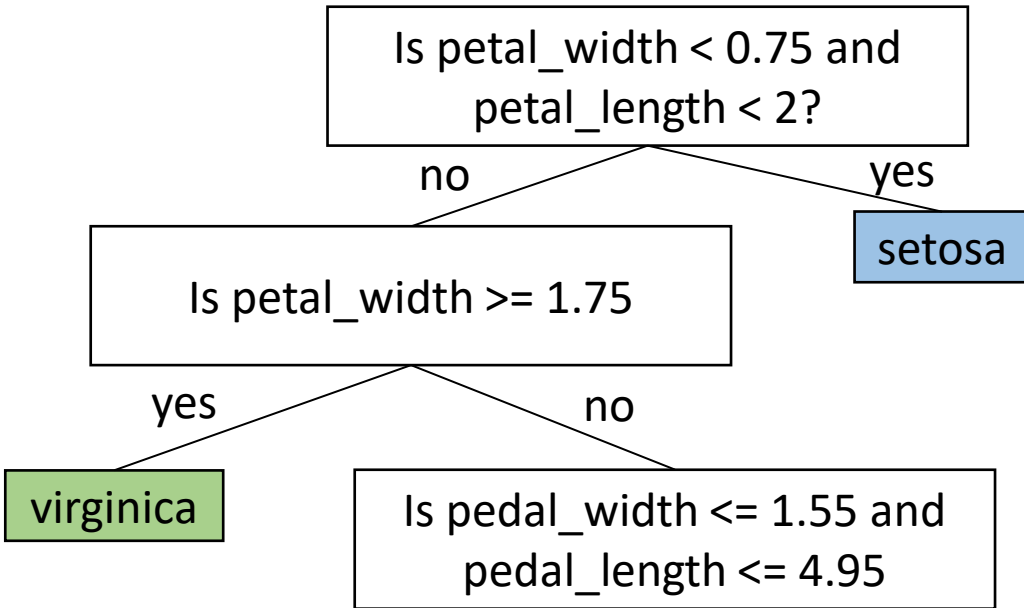
Example: Using Petal Data Only



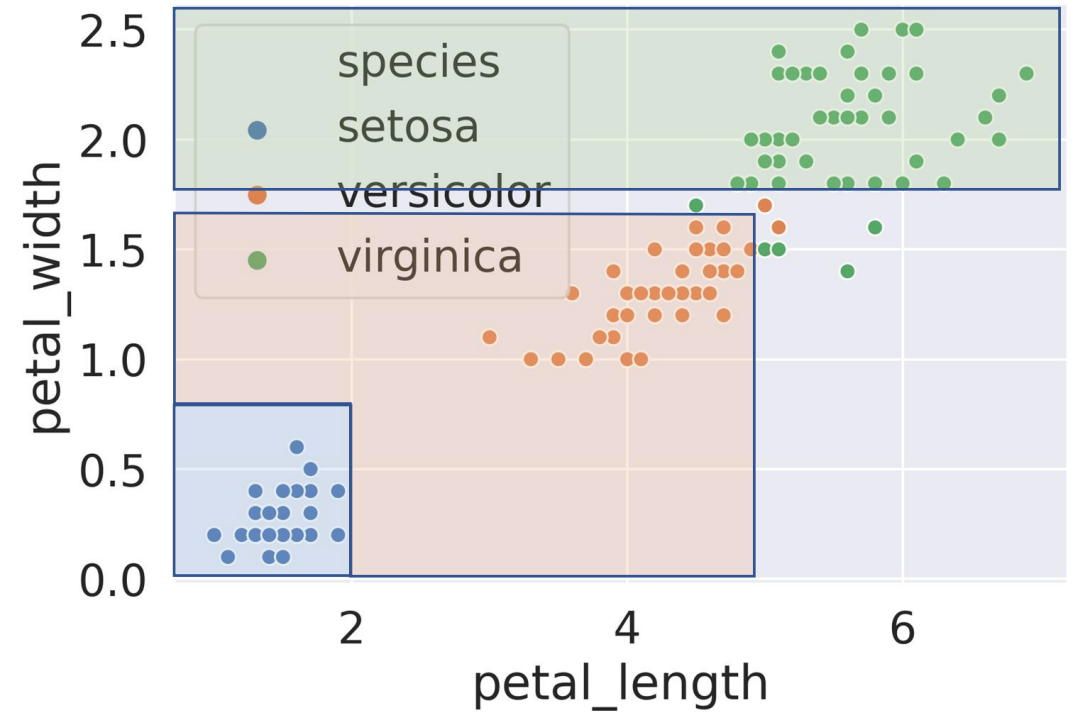
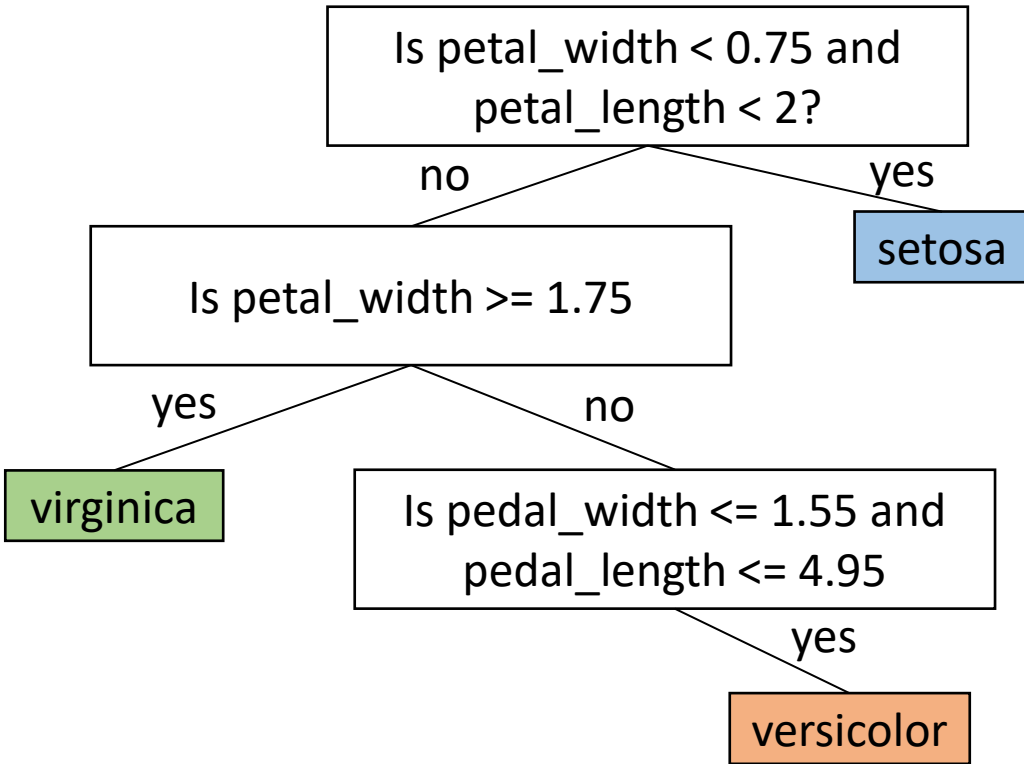
Example: Using Petal Data Only



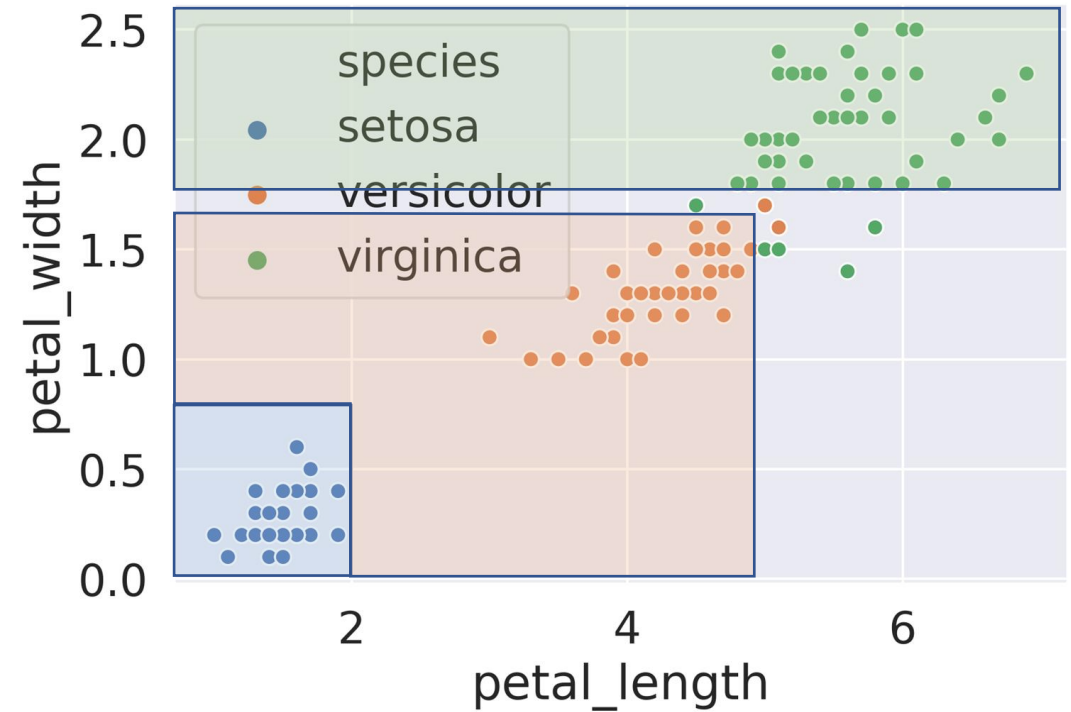
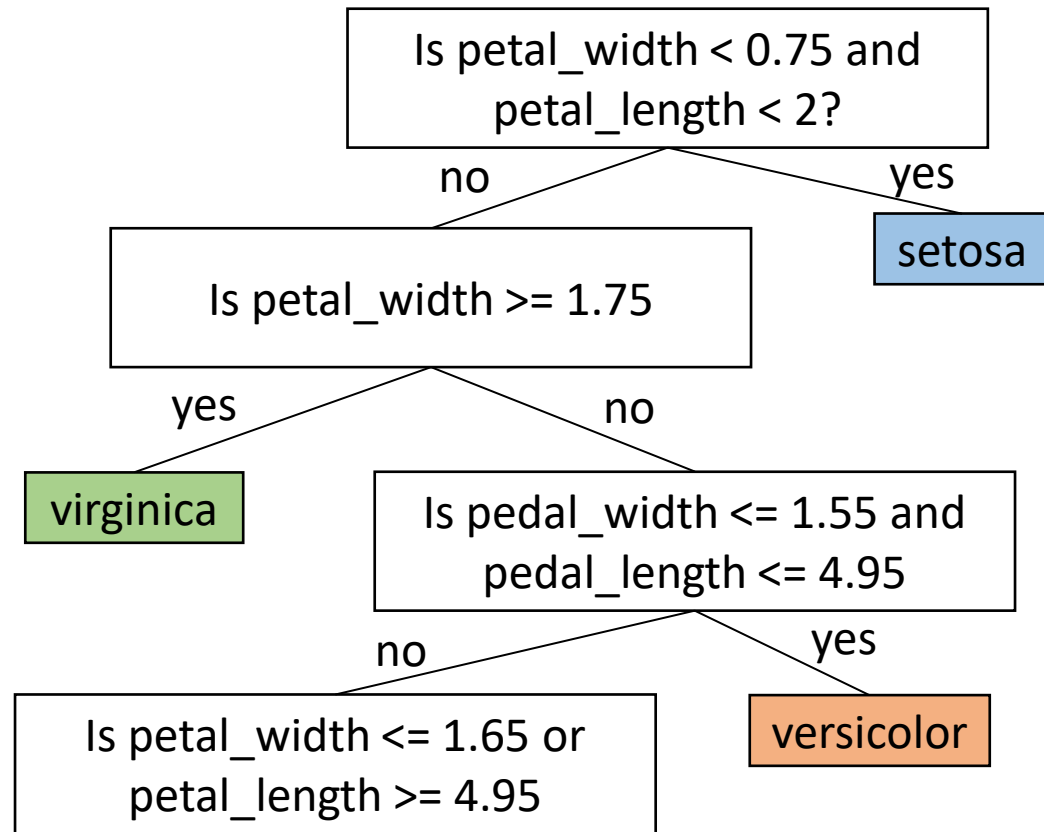
Example: Using Petal Data Only



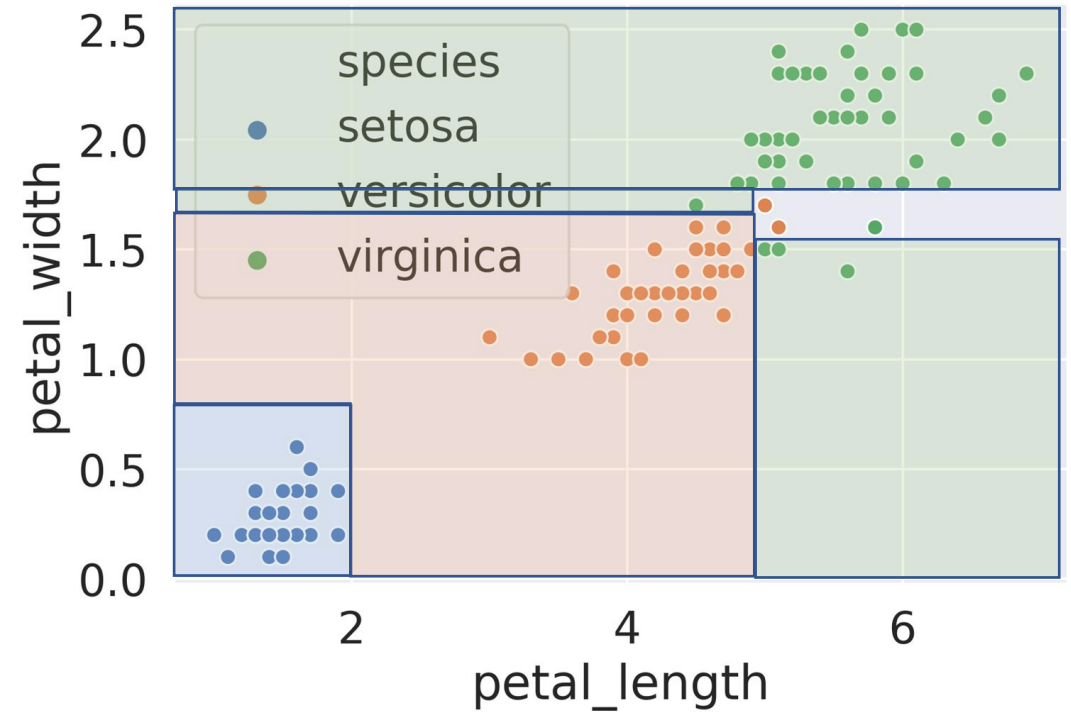
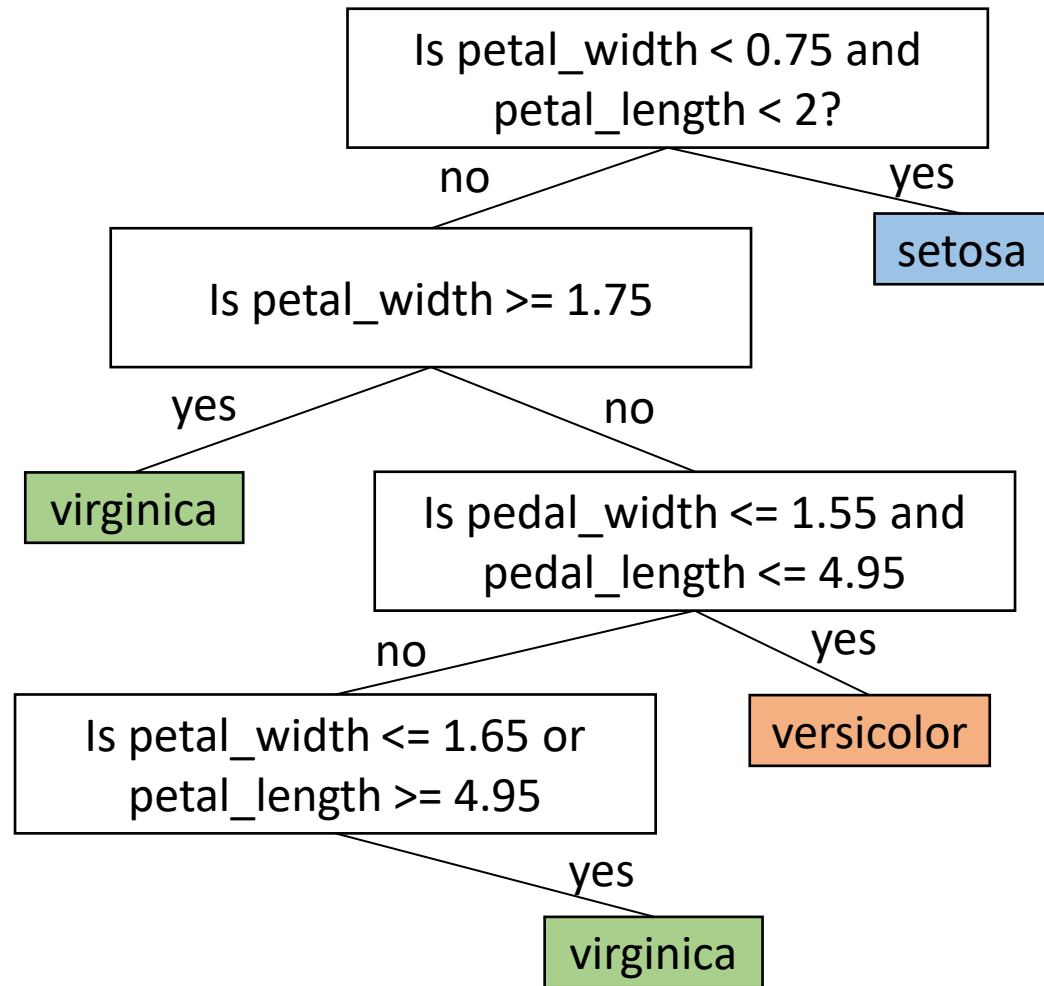
Example: Using Petal Data Only



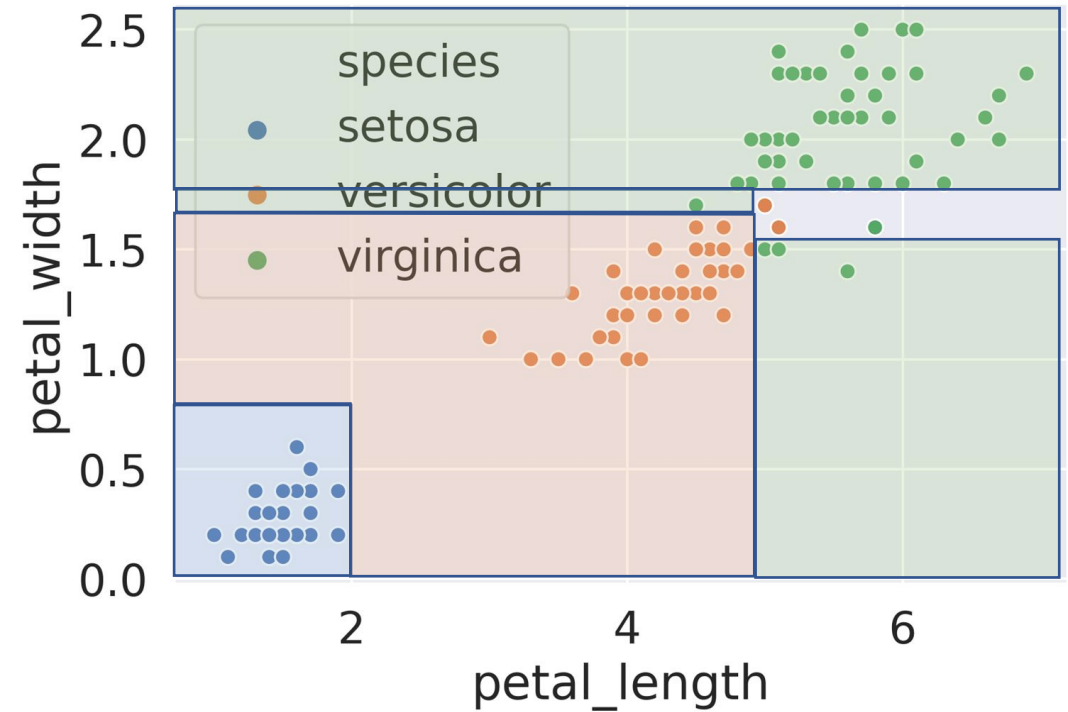
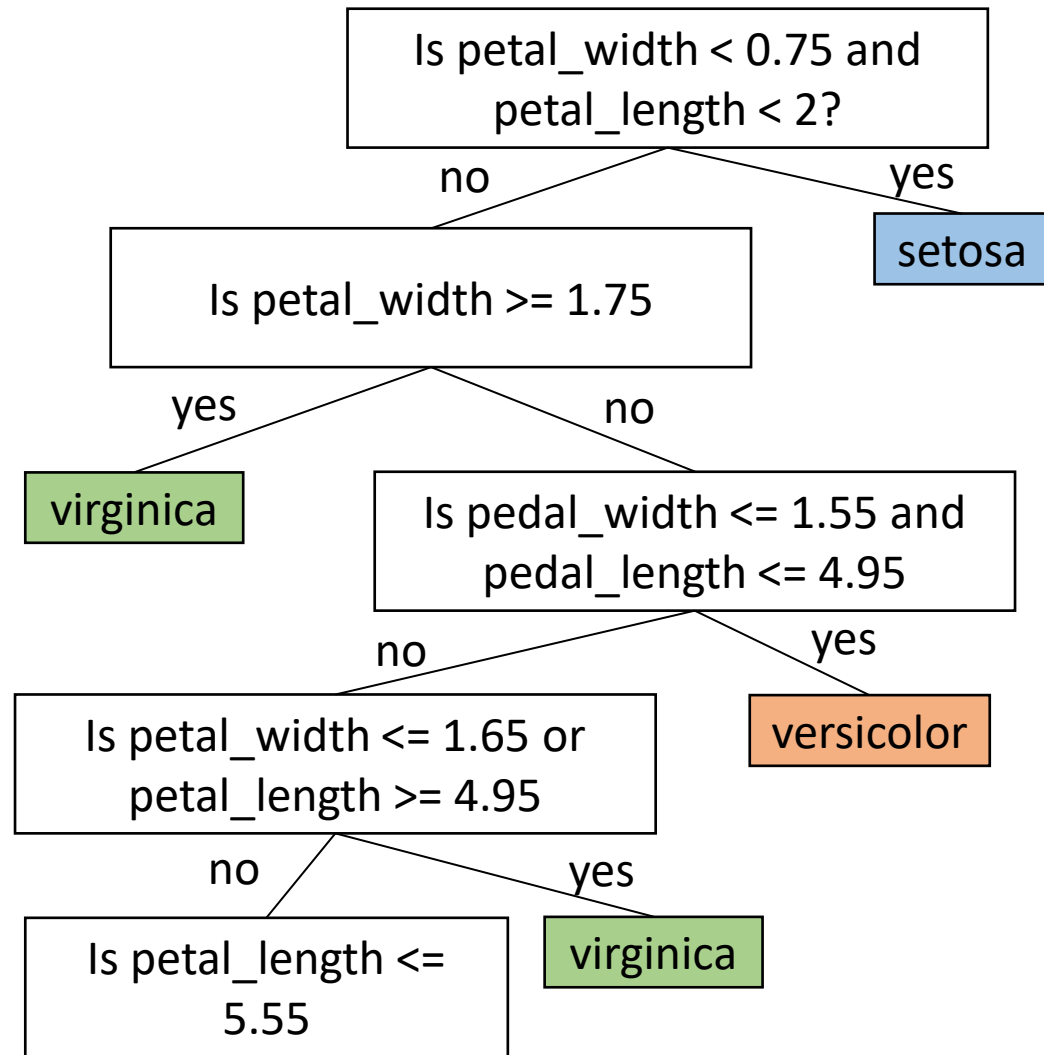
Example: Using Petal Data Only



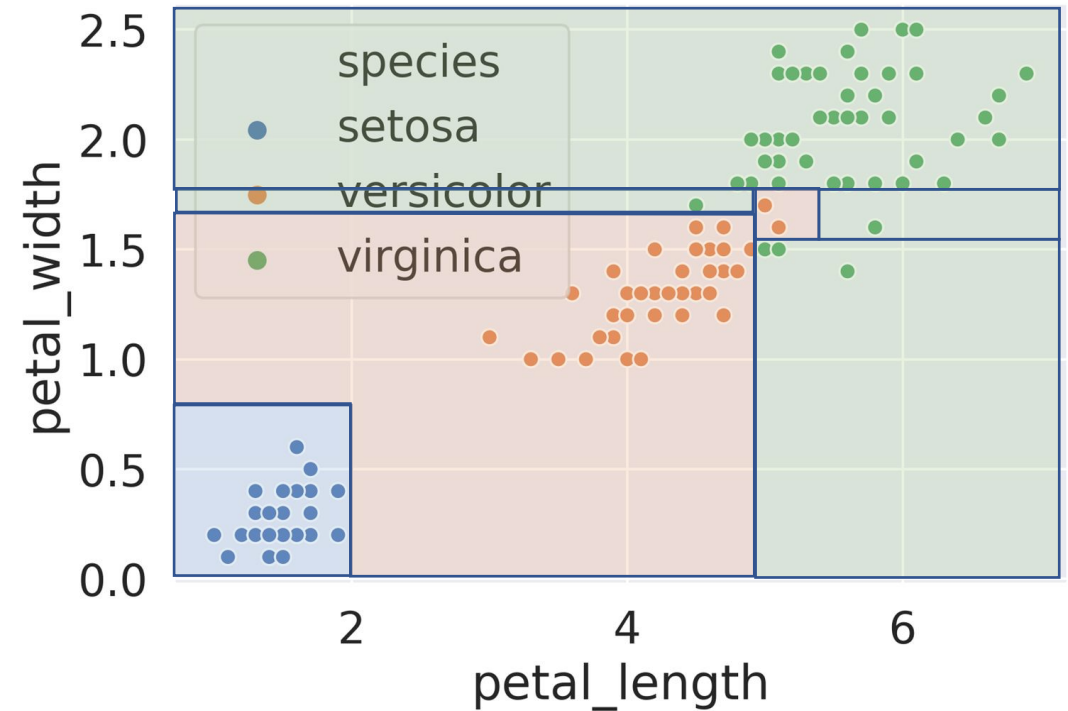
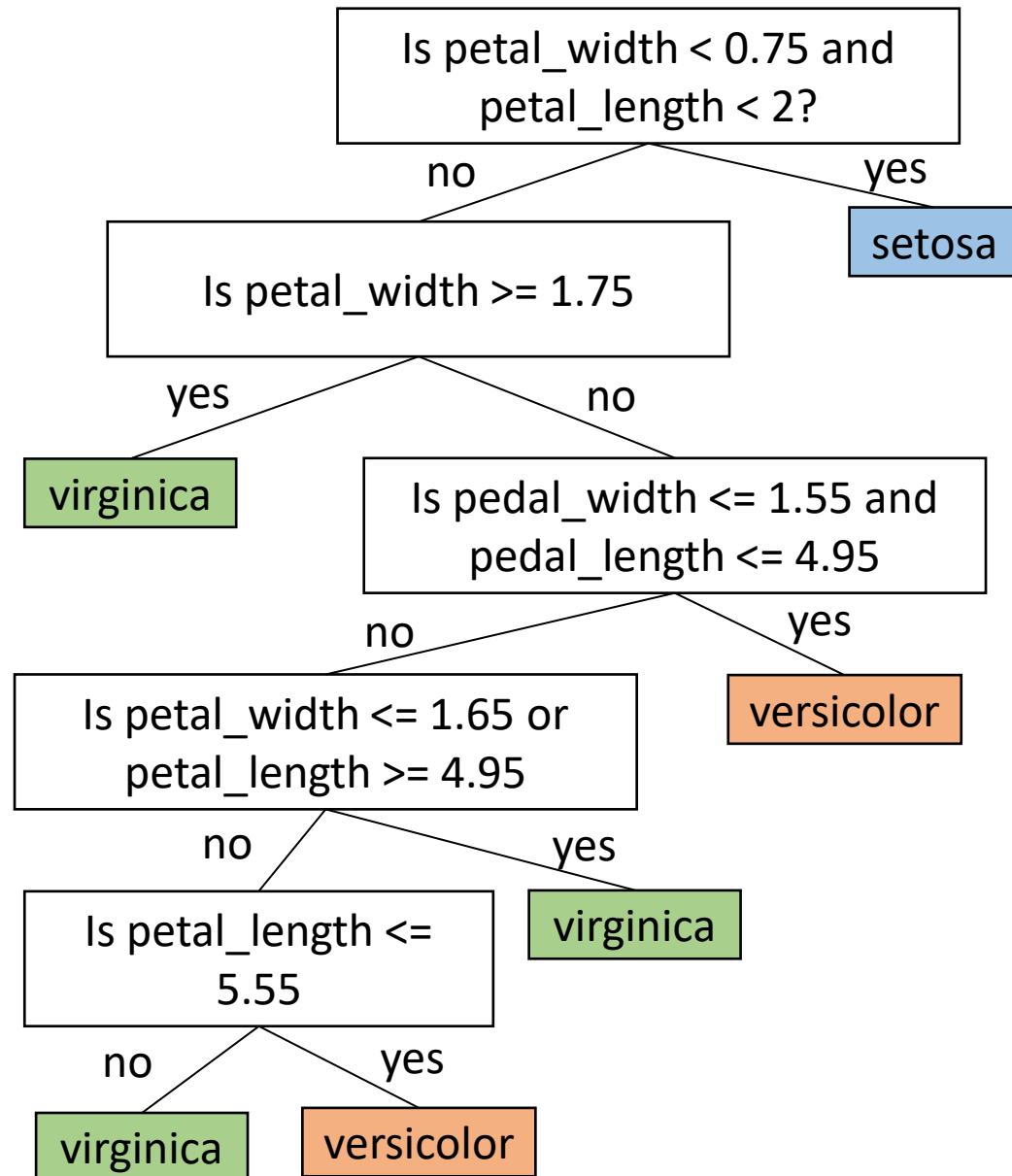
Example: Using Petal Data Only



Example: Using Petal Data Only



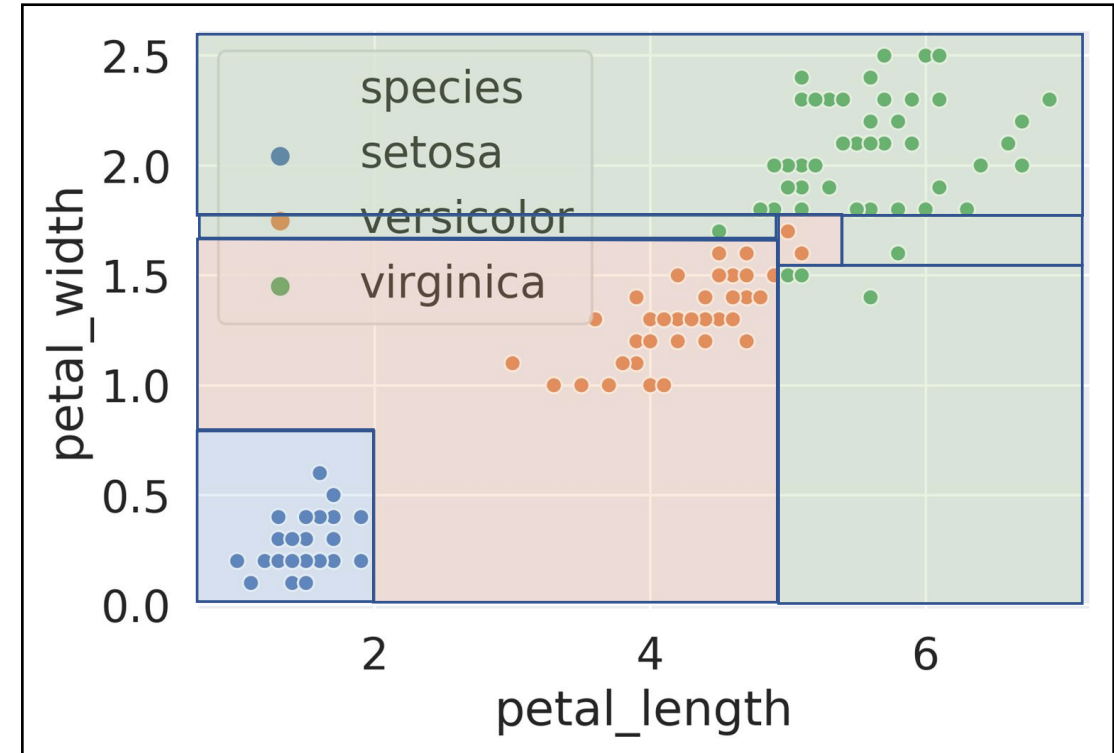
Example: Using Petal Data Only



Example: Using Petal Data Only

How accurate is our decision tree model on the training data?

Is this good or bad?



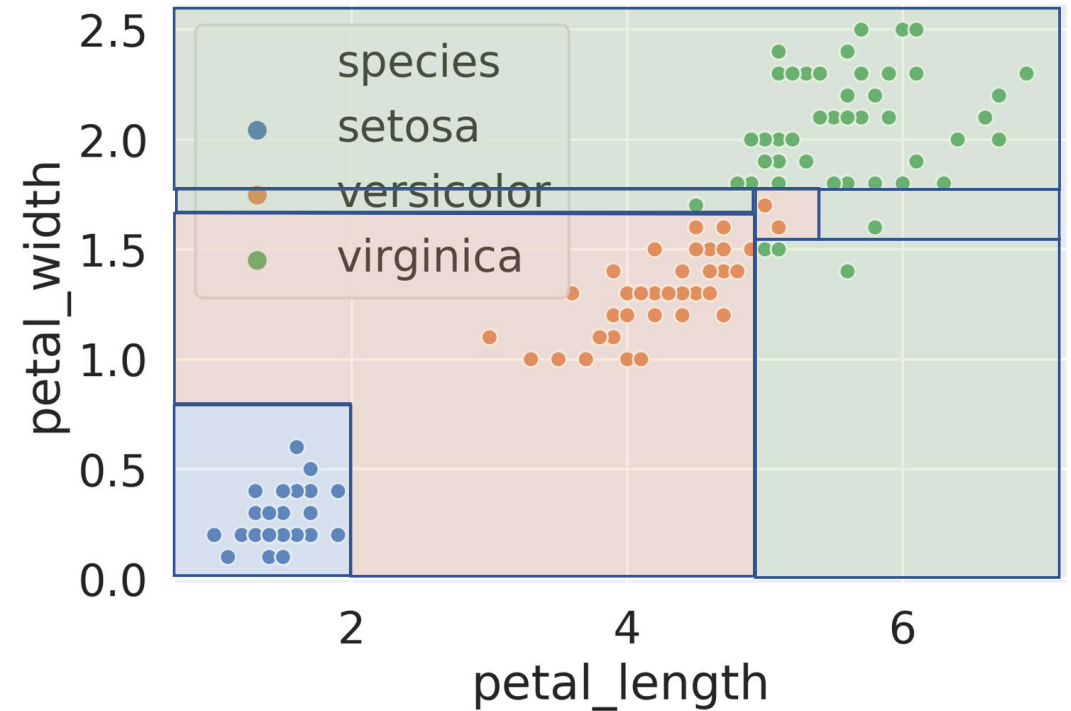
Example: Using Petal Data Only

How accurate is our decision tree model on the training data?

- It seems like it gets every point correct.

Is this good or bad?

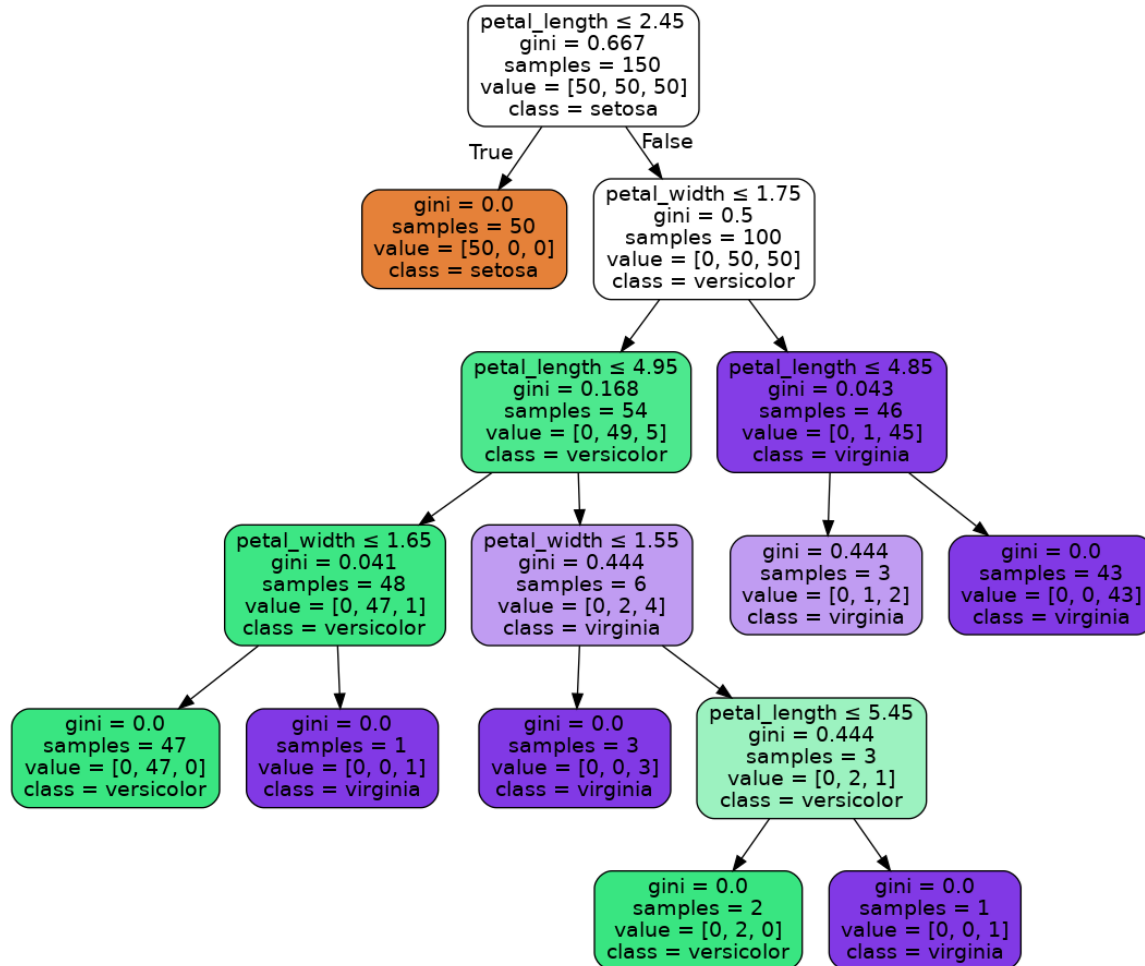
- I'd argue bad.
- Seems likely to result in overfitting!
- Will discuss overfitting more later.



First, let's see how we can build decision trees for classification using scikit-learn.

Visualizing Decision Tree Models

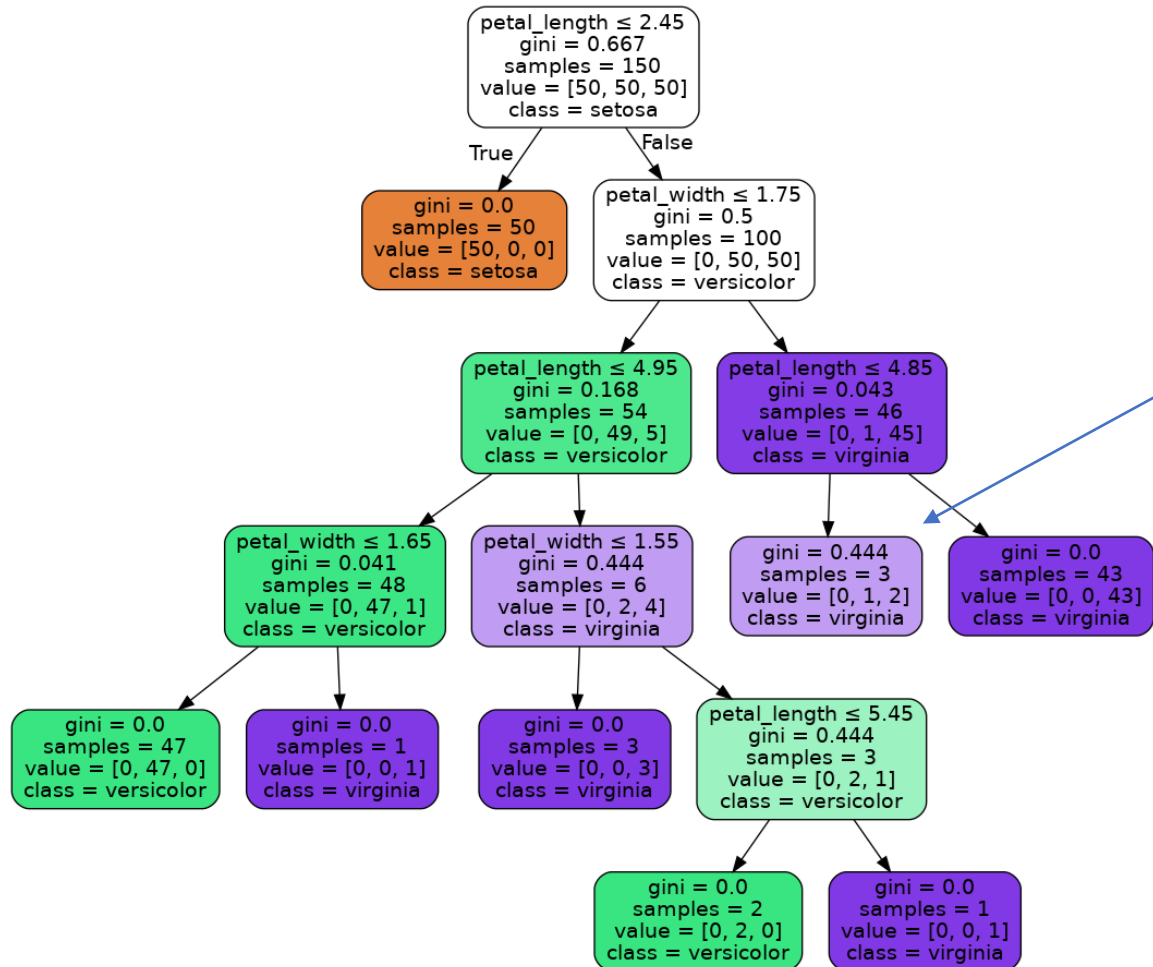
```
from sklearn import tree
decision_tree_model = tree.DecisionTreeClassifier()
decision_tree_model = decision_tree_model.fit(iris_data[["petal_length", "petal_width"]], iris_data["species"])
```



In each box, we see:

- The rule.
- The gini impurity (chance that a sample would be misclassified if randomly assigned at this point).
- The number of samples still unclassified.
- The number of samples in each class still unclassified.
- The most likely class.

Understanding Our Decision Tree



There is one terminal decision point where there is more than one possible right answer.

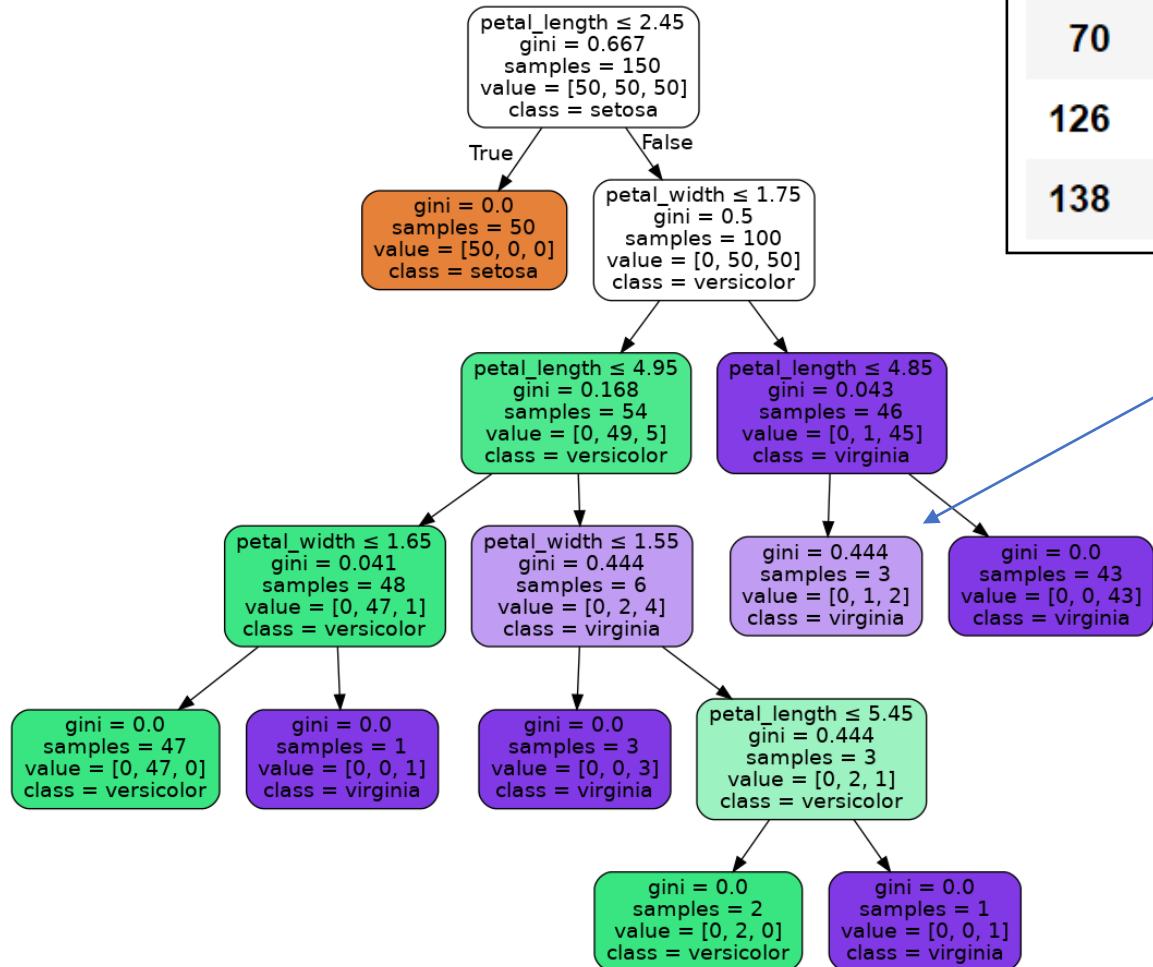
The model was unable to come up with a decision rule to resolve these last 3 samples.

Let's see why using the query method of the dataframe class.

Understanding Our Decision Tree

```
iris_data.query("petal_length > 2.45 and petal_width > 1.75 and petal_length < 4.85")
```

	sepal_length	sepal_width	petal_length	petal_width	species
70	5.9	3.2	4.8	1.8	versicolor
126	6.2	2.8	4.8	1.8	virginica
138	6.0	3.0	4.8	1.8	virginica



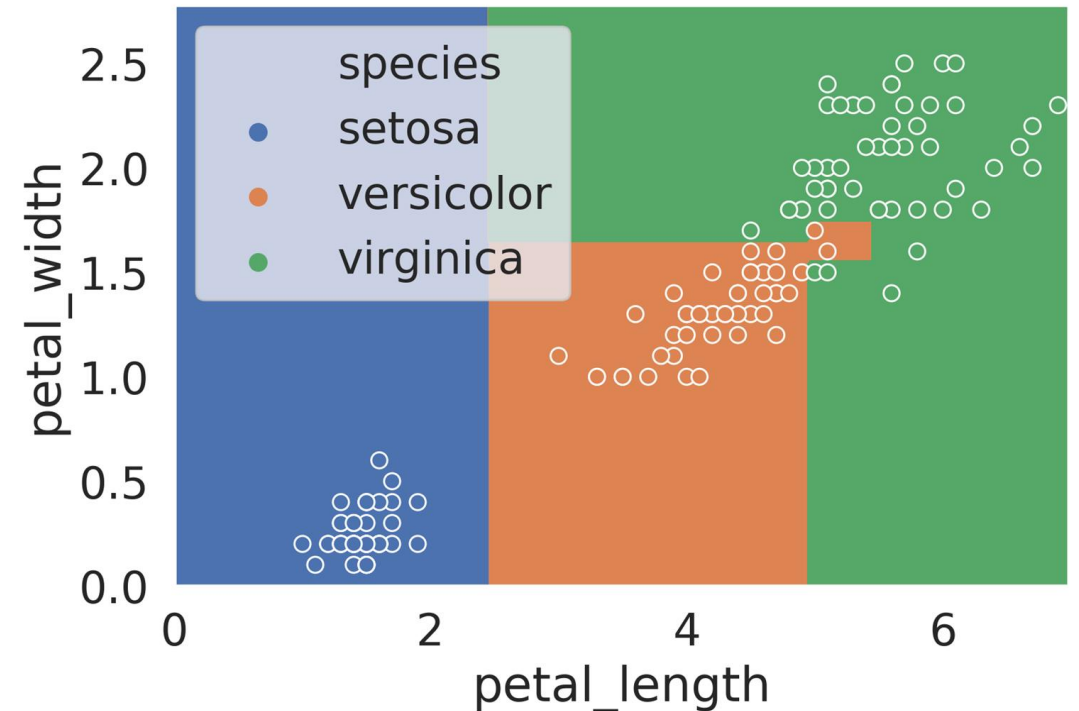
There is one terminal decision point where there is more than one possible right answer.

- In the original data set, there was a versicolor iris with the same petal measurements as two virginicas.

Visualizing Decision Tree Models

Plotting the decision boundaries for our model:

- Decision tree has nonlinear boundary



Measuring the Performance of Our Model

Running the code below, we see that we only get 99.3% accuracy.

```
from sklearn.metrics import accuracy_score
predictions = decision_tree_model.predict(iris_data[["petal_length", "petal_width"]])
accuracy_score(predictions, iris_data["species"])
```

To understand why, let's look back at our decision tree model.

Overfitting and Decision Trees

scikit-learn makes it easy to generate decision trees.

	sepal_length	sepal_width	petal_length	petal_width	species
70	5.9	3.2	4.8	1.8	versicolor
126	6.2	2.8	4.8	1.8	virginica
138	6.0	3.0	4.8	1.8	virginica

Perfect accuracy on the training data, EXCEPT when there are samples from different categories with the exact same features.

- That is, if the versicolor above had a petal_length of 4.800001, we'd have 100% training accuracy.

-> This tendency for perfect accuracy should give us concern about overfitting. (We will go over this later!)

Multidimensional Decision Trees

Naturally, we can include even more features. For example, if we want to use the petal AND sepal measurements, we simply train the decision tree on all four columns of the data.

```
decision_tree_model_4d = tree.DecisionTreeClassifier()  
decision_tree_model_4d = decision_tree_model_4d.fit(train_iris_data[["petal_length", "petal_width",  
                                                                    "sepal_length", "sepal_width"]], train_iris_data["species"])
```

The resulting model gets:

- 100% accuracy on the training set (no overlapping data points).
- 95% accuracy on the test set.

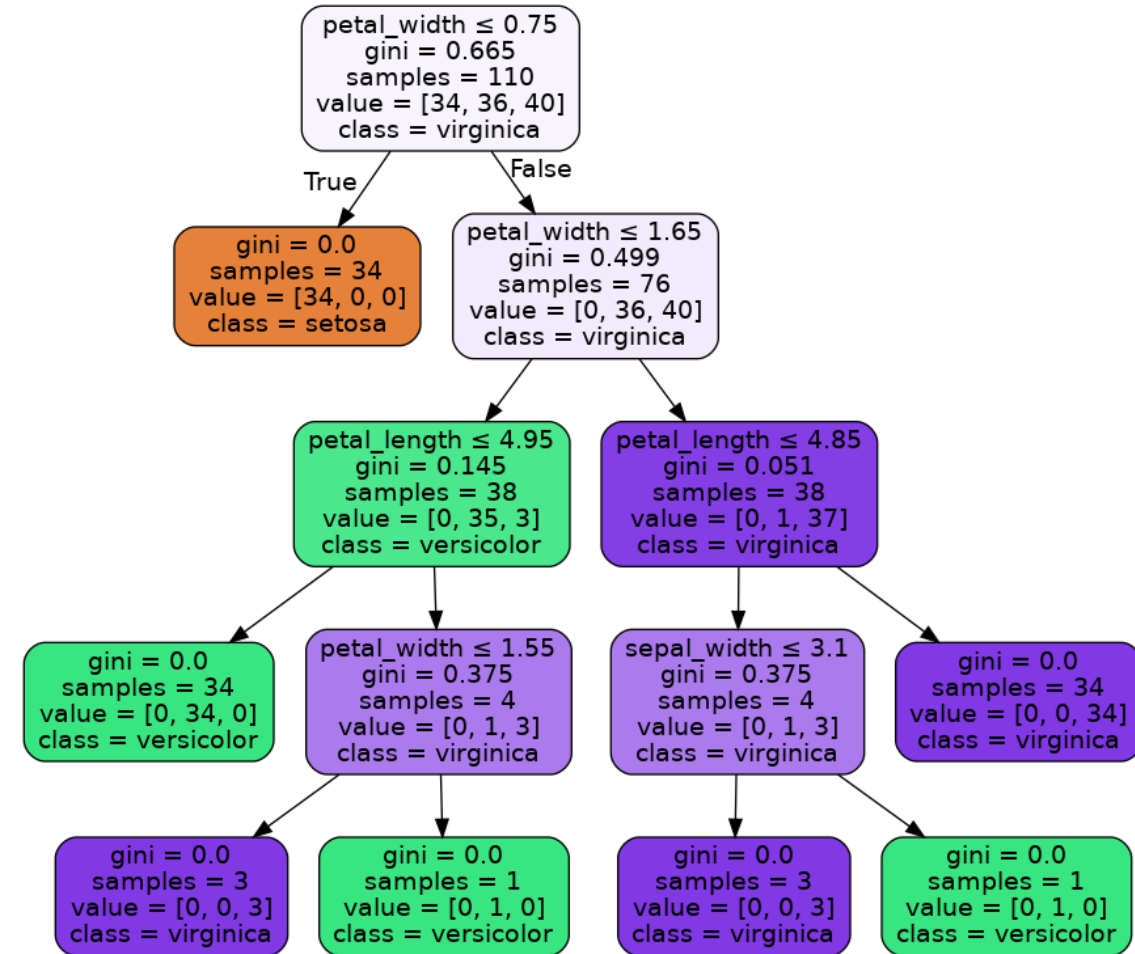
Model 2D-150 vs. Model 4D-110 Tree Visualization

Models are quite similar:

- Model 4D-110 uses only 110 samples.
- The decision rules are nearly identical, except that Model4D-110 **uses the sepal_width exactly once to resolve the case that we couldn't resolve with petal_length and petal_width.**

Model 4D-110 seems marginally better, but both got only 95% accuracy on test set.

- Need more data to know for sure.



Decision Tree Generation

Let's discuss how decision trees are created from data.

Traditional decision tree generation algorithm:

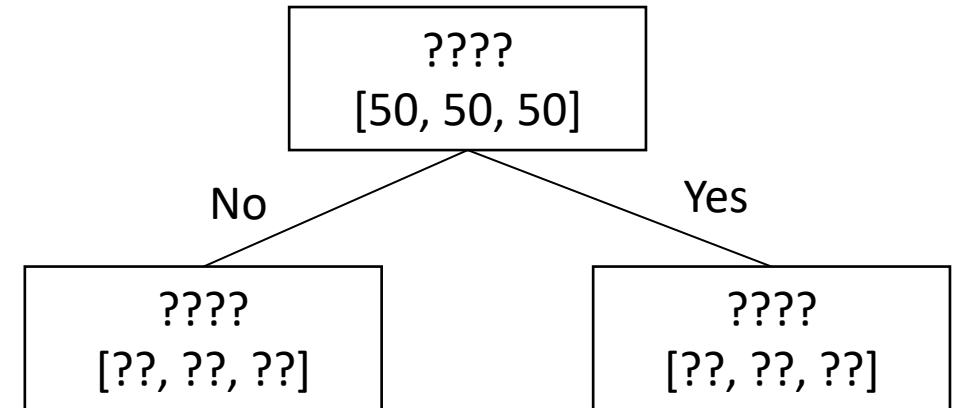
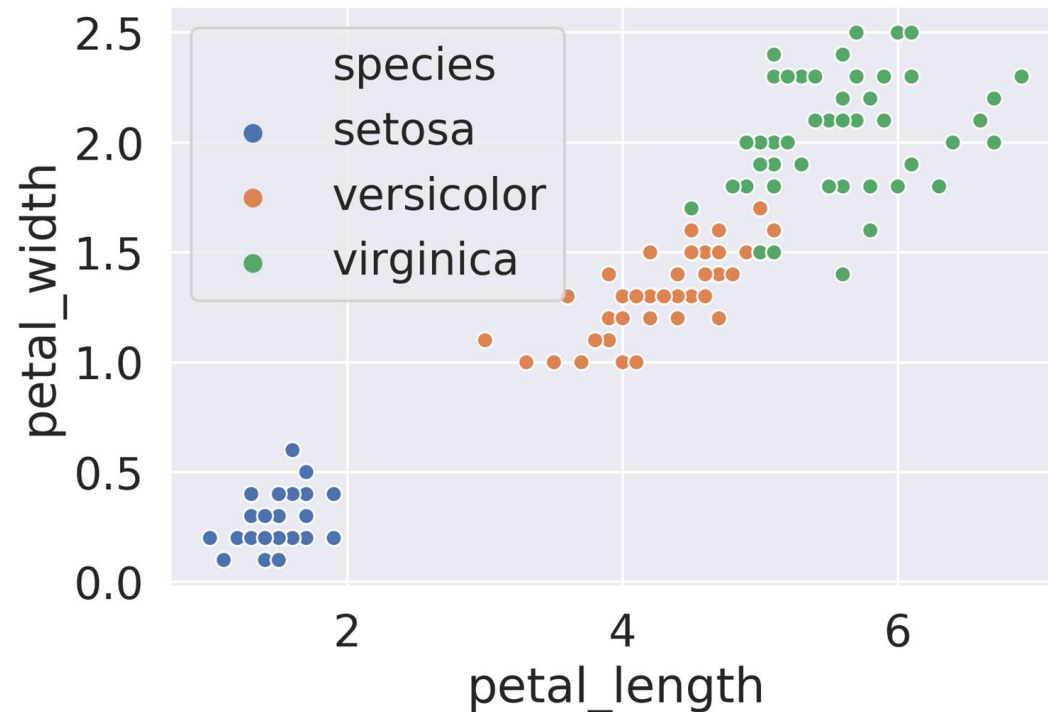
- All of the data starts in the root node.
- **Repeat** until every node is either **pure** or **unsplittable**:
 - **Pick the best feature** x and **best split value** β , e.g. $x = \text{petal_length}$, $\beta = 2$.
 - **Split data into two nodes**, one where $x < \beta$, and one where $x \geq \beta$.

Notes: A node that has only one type is called a “**pure**” node. A node that has duplicate data that cannot be split is called “**unsplittable**”.

Defining a Best Feature

Question: Which feature and split value is best?

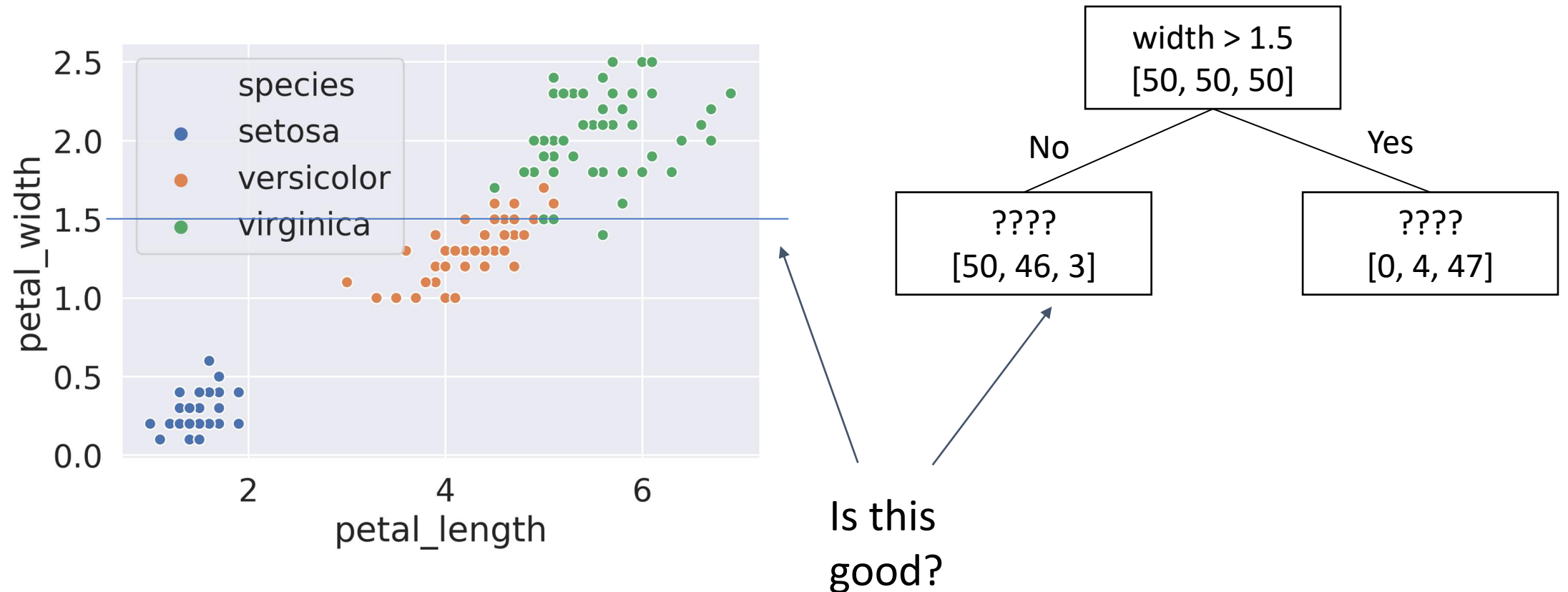
- Equivalently: Which horizontal or vertical line do we want to draw?



Defining a Best Feature

Question: Which feature and split value is best?

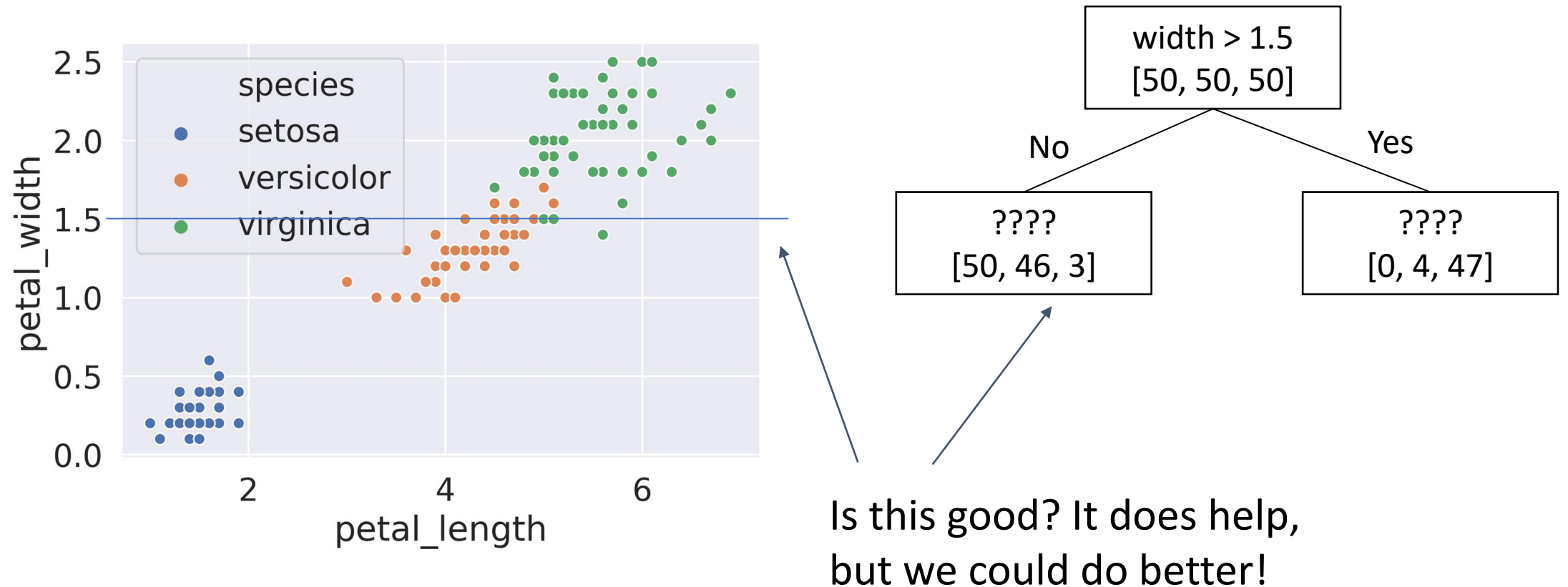
- Equivalently: Which horizontal or vertical line do we want to draw?



Defining a Best Feature

Question: Which feature and split value is best?

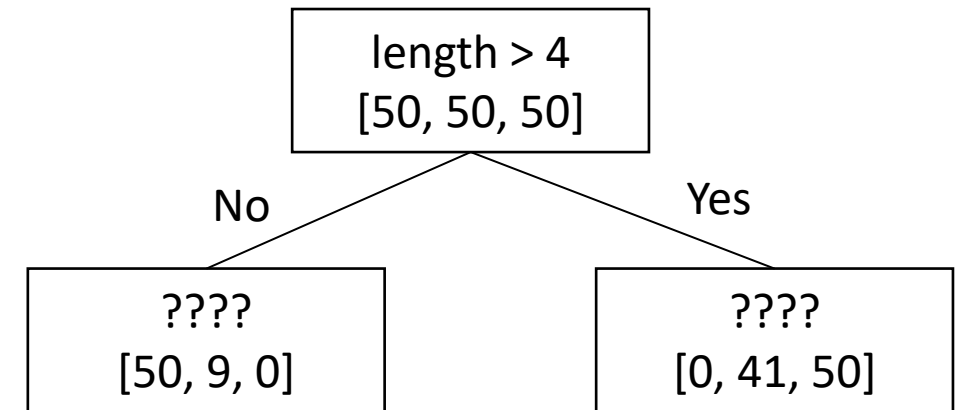
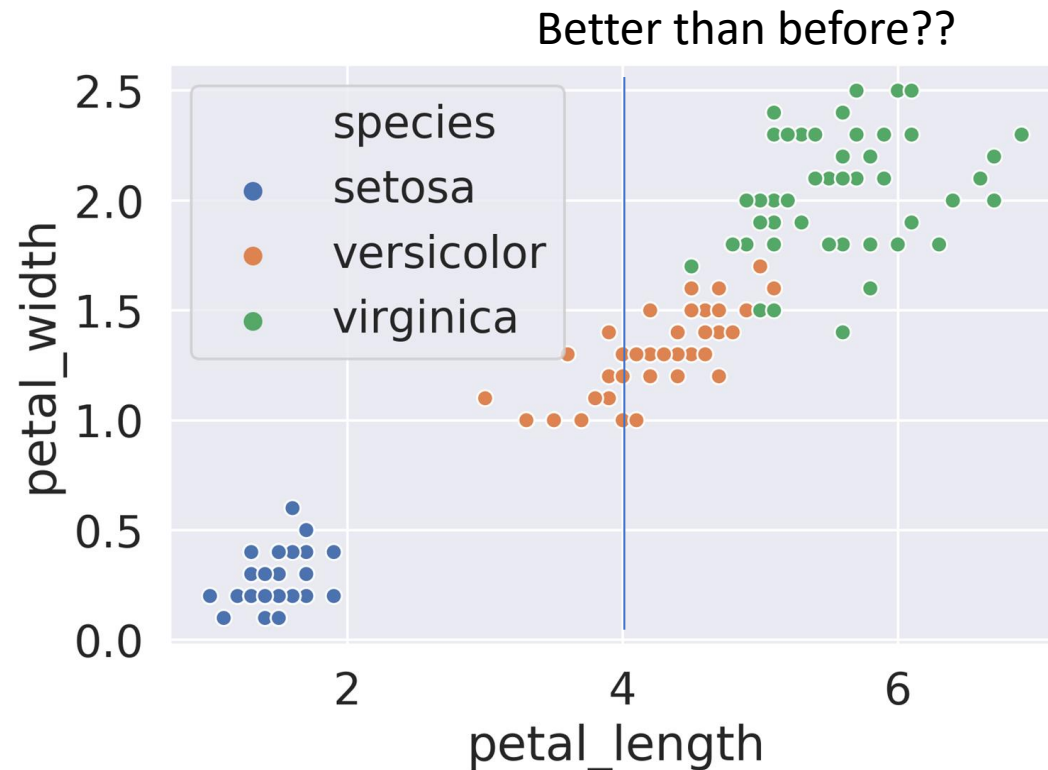
- Equivalently: Which horizontal or vertical line do we want to draw?



Defining a Best Feature

Question: Which feature and split value is best?

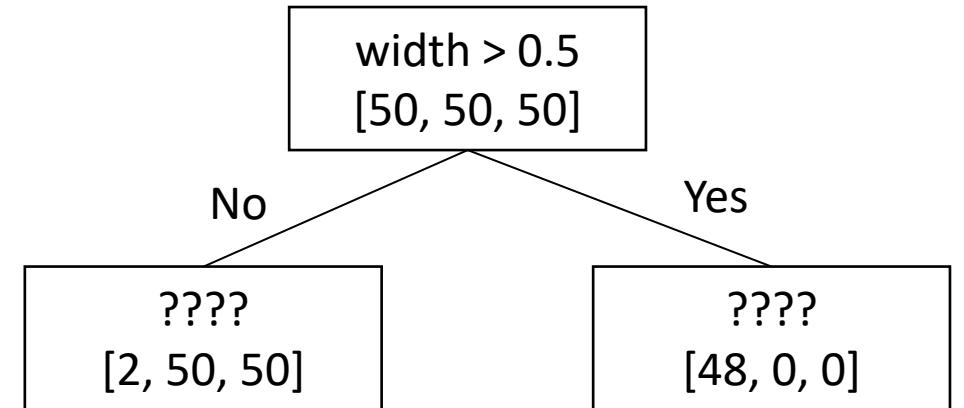
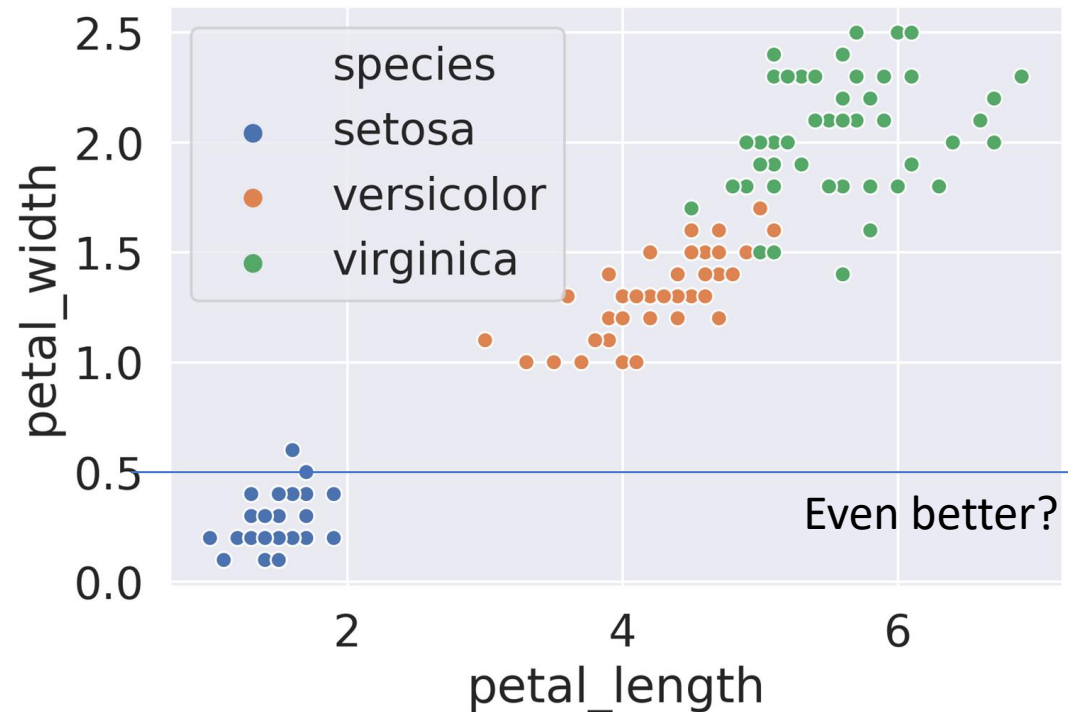
- Equivalently: Which horizontal or vertical line do we want to draw?



Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?

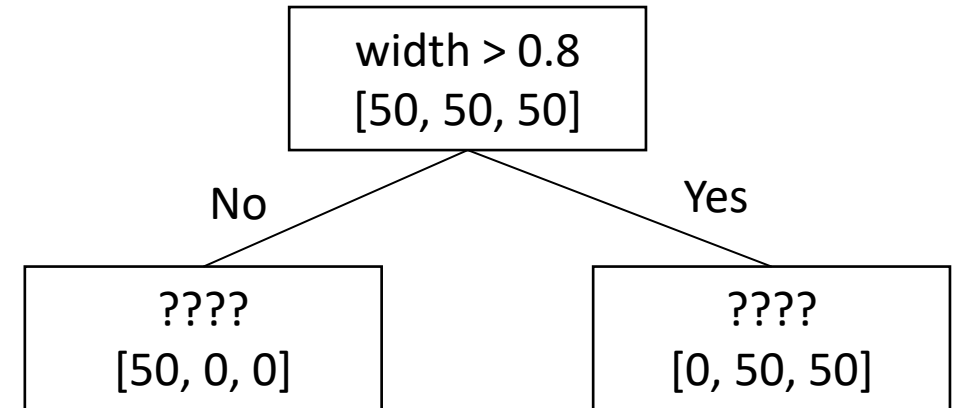
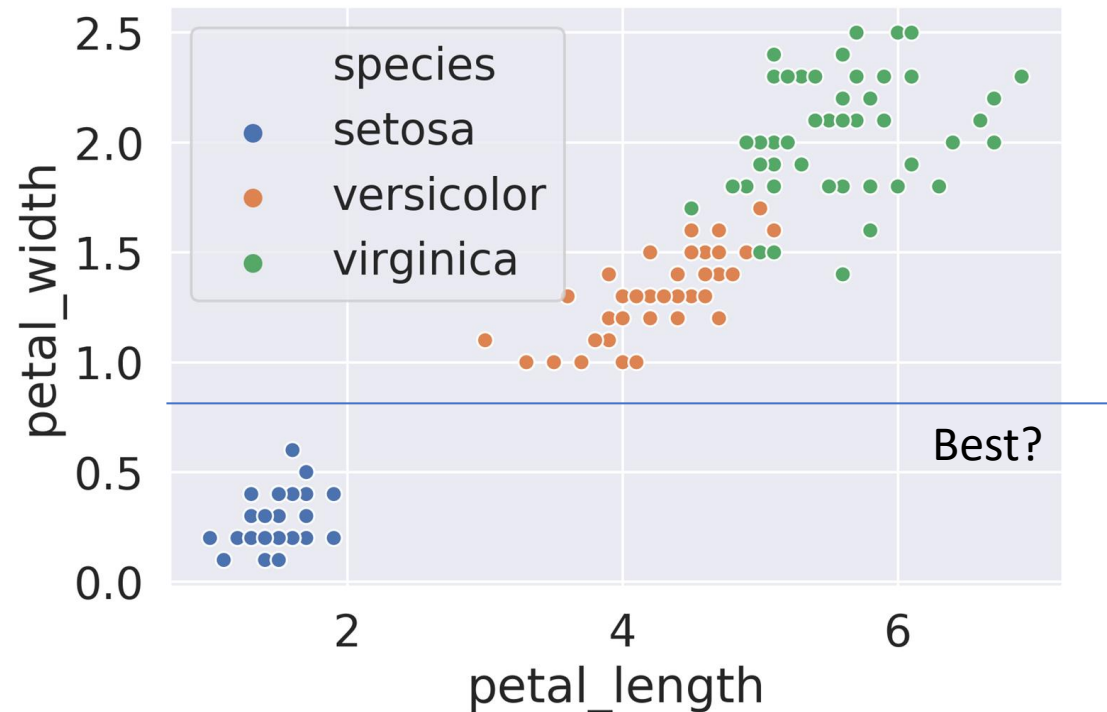


Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?

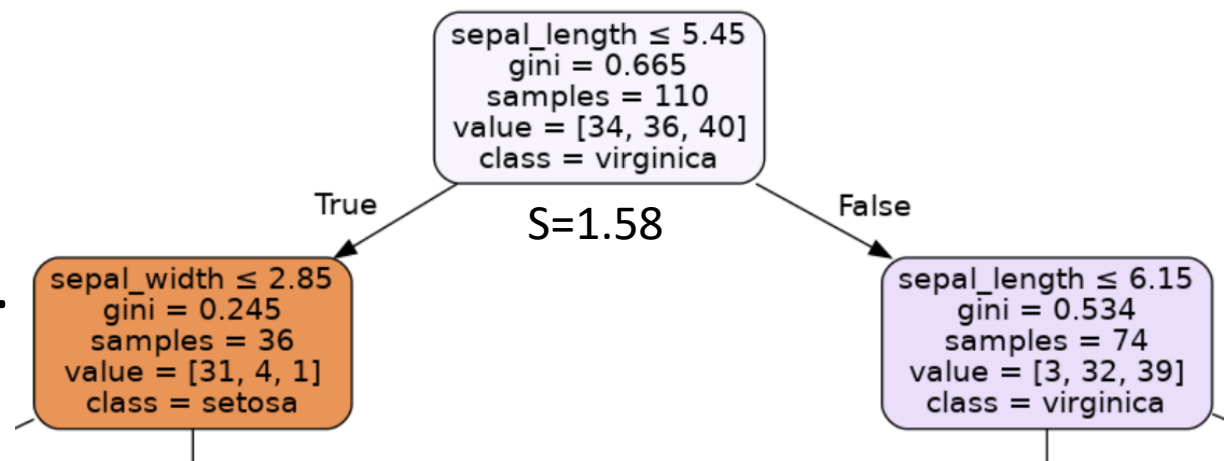
We need some sort of rigorous definition for a good split.



Node Entropy

Let p_C be the proportion of data points in a node with label C.

For example, for the node at the top of the decision tree, $p_0 = 34/110 = 0.31$, $p_1 = 36/110 = 0.33$, and $p_2 = 40/110 = 0.36$.



Define the entropy S of a node as:

$$S = - \sum_C p_C \log_2 p_C$$

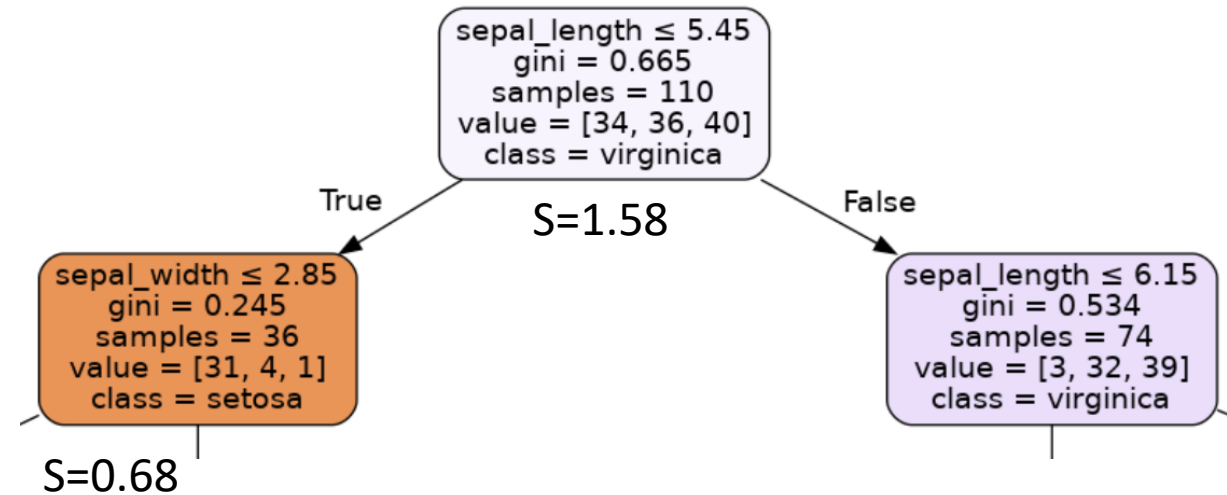
For example, S for the top node is:

$$-0.31 \log_2 0.31 - 0.33 \log_2 0.33 - 0.36 \log_2 0.36 = 0.52 + 0.53 + 0.53 = 1.58$$

Test Your Understanding

What is the entropy of the node on the left with [31, 4, 1] in each class?

- $p_0 = 31/36 = 0.86$, $p_1 = 4/36 = 0.11$, and $p_2 = 1/36 = 0.028$
- $S = -0.86 \log_2 0.86$
– $0.11 \log_2 0.11$
– $0.028 \log_2 0.028 = 0.68$



Define the entropy S of a node as:

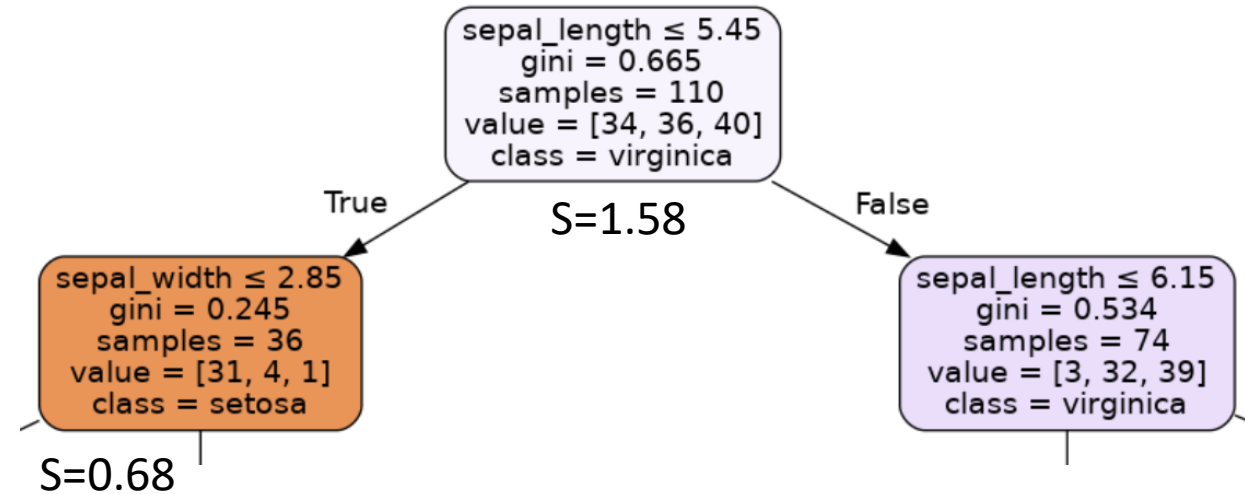
$$S = - \sum_c p_c \log_2 p_c$$

Can think of entropy as how unpredictable a node is. Low entropy means more predictable. High entropy means more unpredictable.

Exploring Entropy

Observations about entropy:

- A node where all data are part of the same class has zero entropy.
 $-1 \log_2 1 = 0$
- A node where data are evenly split between two classes has entropy 1.
 $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$
- A node where data are evenly split between 3 classes has entropy 1.58.
 $3 \times (-0.33 \log_2 0.33) = 1.58$
- A node where data are evenly split into C classes has entropy $\log_2 C$.
 $C \times (-1/C \log_2 1/C) = -\log_2 1/C = \log_2 C$



$$S = - \sum_C p_C \log_2 p_C$$

Weighted Entropy as a Loss Function

We can use Weighted Entropy as a loss function in helping us decide which split to take.

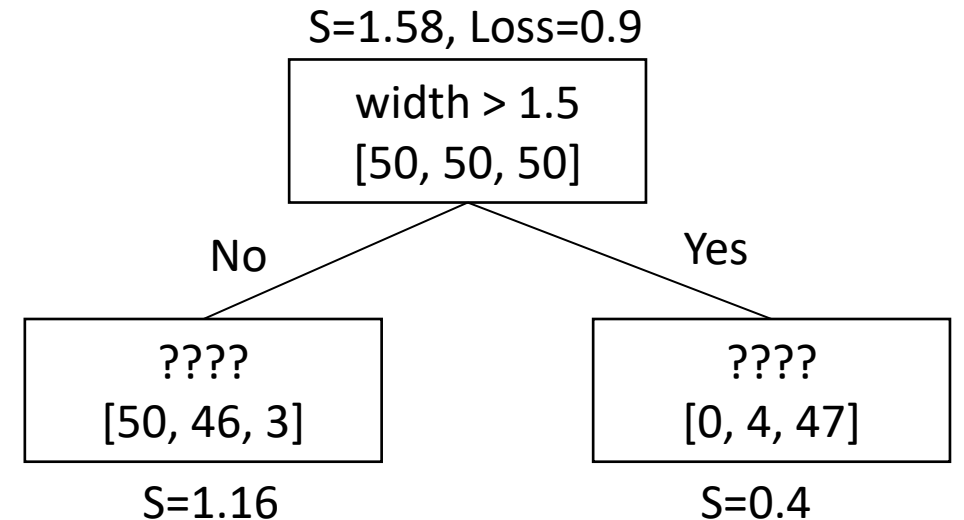
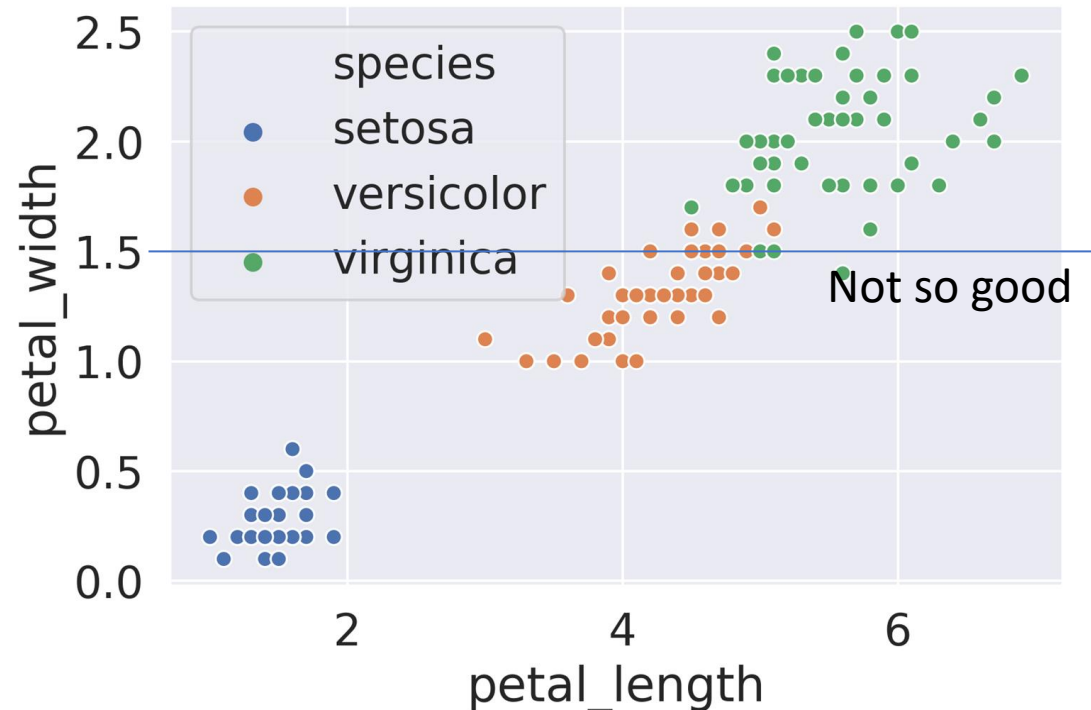
Suppose a given split results in two nodes X and Y with N_1 and N_2 total samples each. The loss of that split is given by:

$$L = \frac{N_1 S(X) + N_2 S(Y)}{N_1 + N_2}$$

Defining a Best Feature

Split choice #1: width > 1.5. Compute entropy of child nodes:

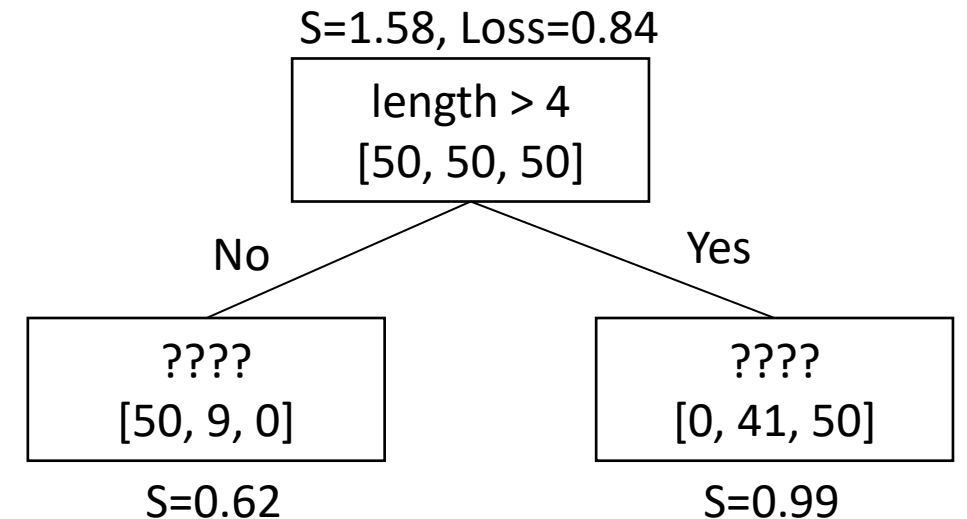
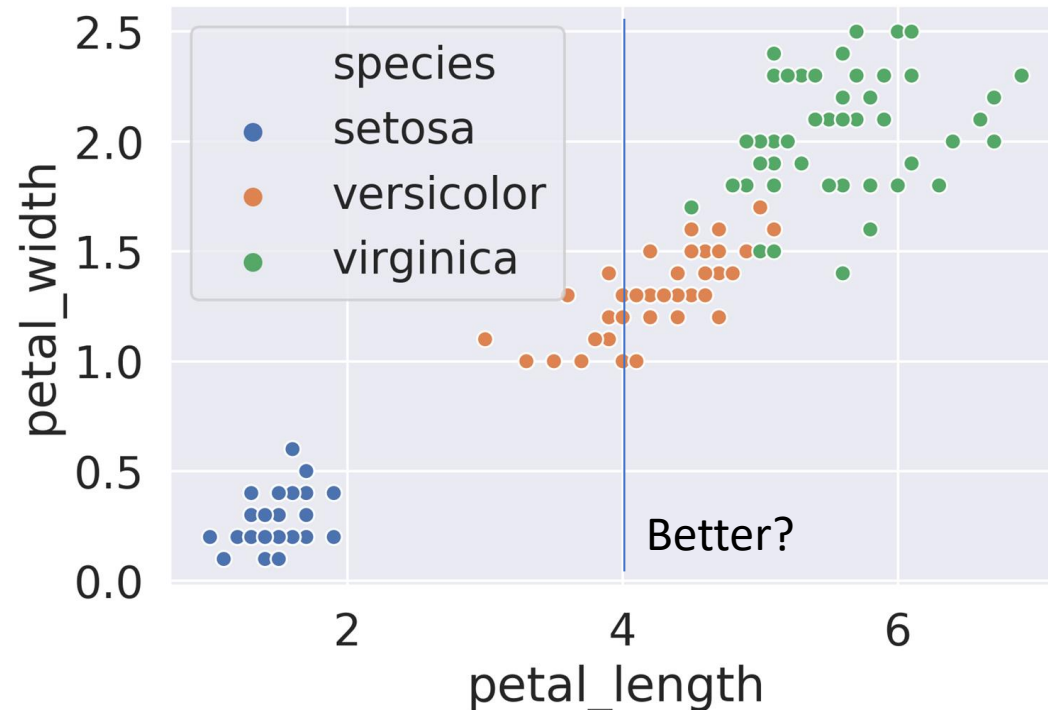
- $\text{entropy}([50, 46, 3]) = 1.16$
- $\text{entropy}([4, 47]) = 0.4$
- Weighted average: $99/150 \times 1.16 + 51/150 \times 0.4 = \mathbf{0.9}$



Defining a Best Feature

Split choice #2: $\text{length} > 4$. Compute entropy of child nodes:

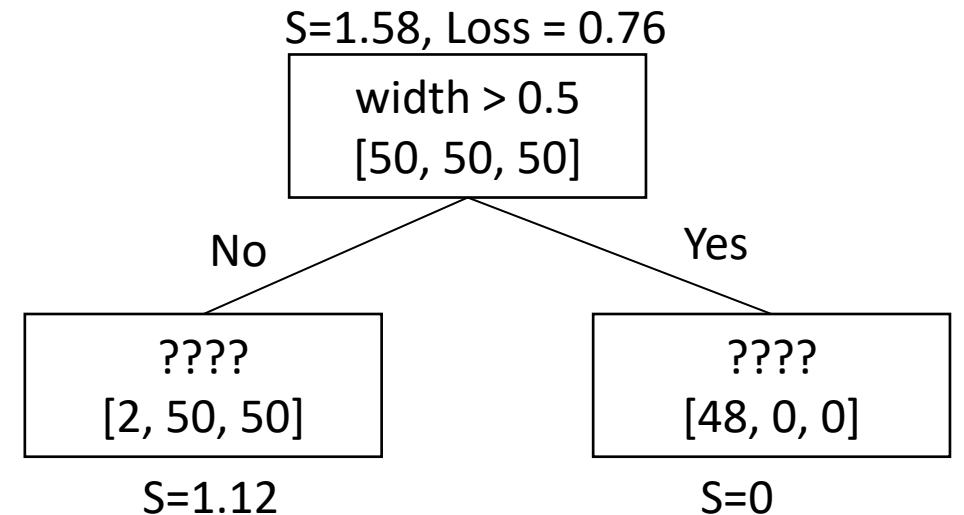
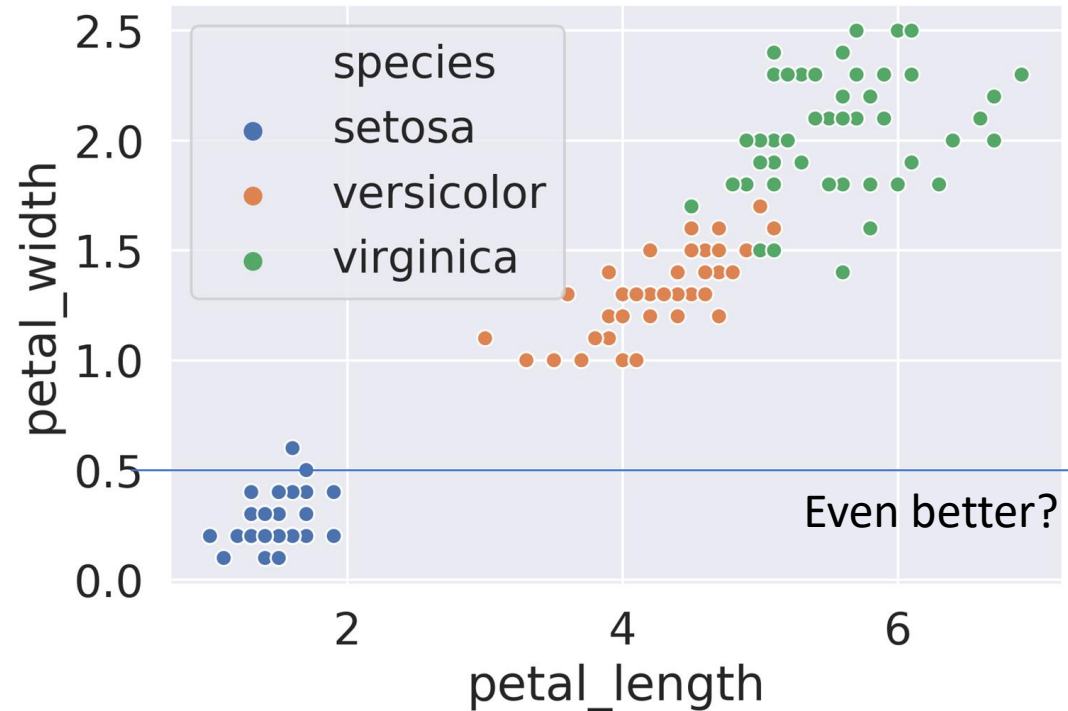
- $\text{entropy}([50, 9]) = 0.62$
- $\text{entropy}([41, 50]) = 0.99$
- Weighted Average: **0.84**: Better than split choice #1!



Defining a Best Feature

Split choice #3: width > 0.5. Compute entropy of child nodes:

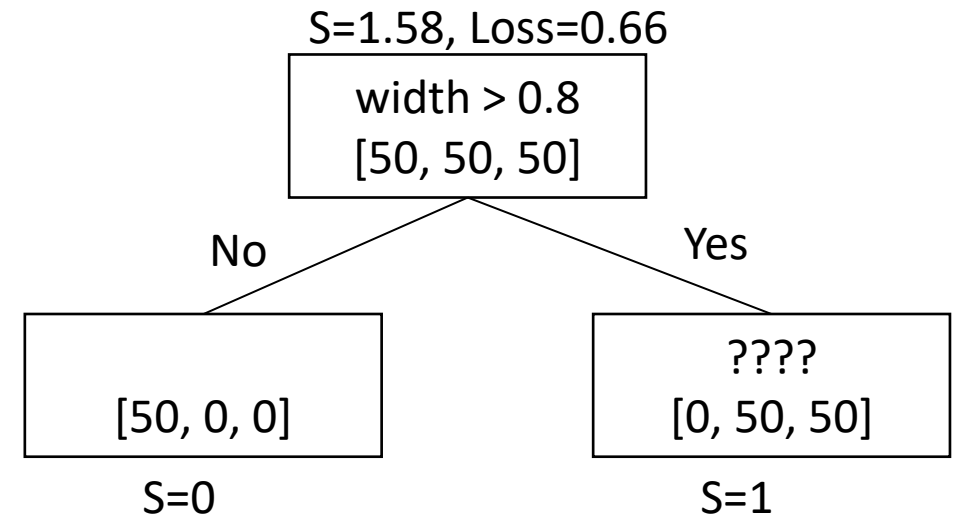
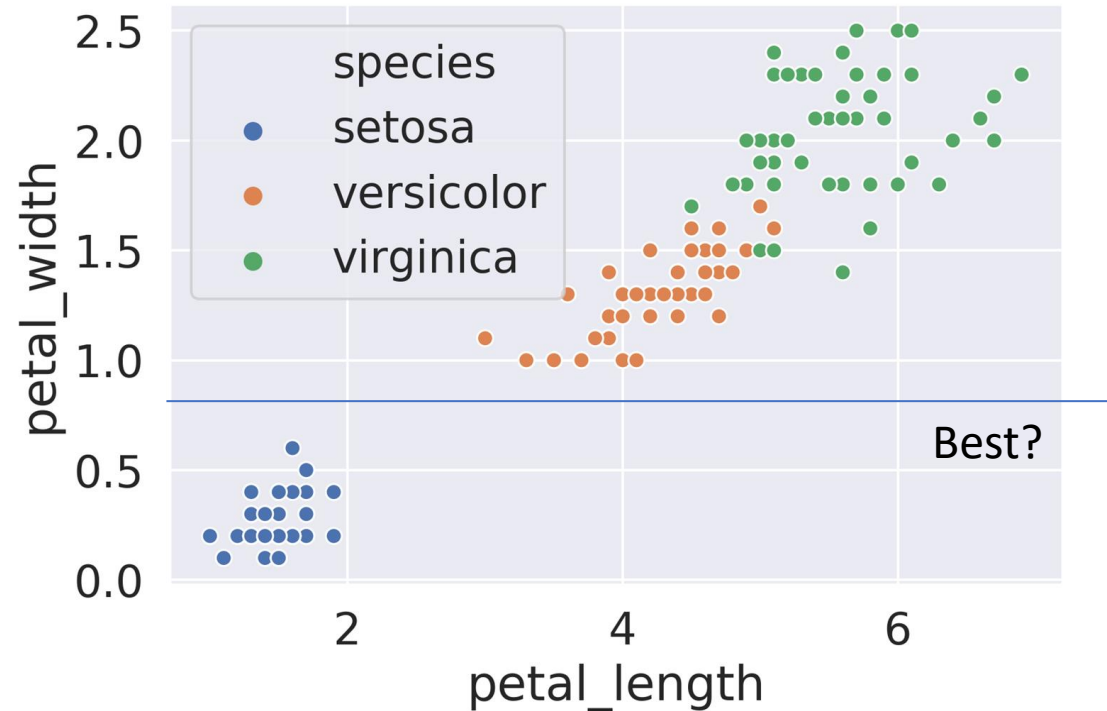
- $\text{entropy}([2, 50, 50]) = 1.12$
- $\text{entropy}([48]) = 0$
- Weighted average: **0.76**: Lower than split choice #2!



Defining a Best Feature

Split choice #4: width > 0.9. Compute entropy of child nodes:

- $\text{entropy}([50, 50]) = 1$
- $\text{entropy}([50]) = 0$
- Weighted average: **0.66**: Lower than split choice #3!



Decision Tree Generation

Traditional decision tree generation algorithm:

- All of the data starts in the root node.
- Repeat until every node is either **pure** or **unsplittable**:
 - **Pick the best feature** x and **split value** β such that the loss of the resulting split is **minimized**, e.g. $x = \text{petal_width}$, $\beta = 0.8$ has loss 0.66.
 - **Split data into two nodes**, one where $x < \beta$, and one where $x \geq \beta$.

Notes: A node that has only one type is called a “**pure**” node. A node that has duplicate data that cannot be split is called “**unsplittable**”.

Let's now turn our attention to avoiding overfitting.

Summary

- Classification
- k-nearest neighbors Algorithms
- Decision Trees
- Building Decision Trees
- Entropy and Defining a Best Feature

Next

- We will revisit the Decision Tree example to learn more about
- Overfitting
- Random Decision Trees
- Regularization

References

Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking by Foster Provost and Tom Fawcett

Introduction to Machine Learning with Python, a Guide for Data Scientists by Andreas C. Müller & Sarah Guido